
PE-Bench: A Benchmark for Evaluating Agent-Based Power Electronics Design Systems

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Power electronics (PE) is essential for electric vehicles, AI data centers,
2 renewable-energy systems, grid inverters, industrial drives, and regulated
3 power supplies. As PE design becomes increasingly complex, agent-based
4 workflows built on large language models, tool use, and multi-agent systems
5 are emerging to automate PE modeling, component selection, optimization,
6 simulation setup, and design-report generation. However, these workflows
7 still lack a systematic evaluation framework. In PE, a qualified design must
8 satisfy performance and safety requirements while maintaining consistency
9 among theoretical derivations, simulations, component ratings, safety mar-
10 gins, and final design claims. Therefore, simple checks on output format or
11 whether a simulation can run are insufficient for evaluating AI-generated
12 PE designs.

13 To address this gap, we propose PE-Bench, a comprehensive and reusable
14 evaluation framework for agent-based PE design systems. PE-Bench consists
15 of a benchmark task suite and a systematic evaluation workflow, referred to
16 as the evaluator throughout the paper. The task suite contains 78 executable
17 benchmark tasks across five representative converter families: Buck, Boost,
18 Buck-Boost, Flyback, and three-phase inverter designs, covering both DC-
19 DC and DC-AC conversion scenarios. Each task includes a natural-language
20 design request, a machine-readable specification, and a feasible reference
21 design, enabling end-to-end evaluation from requirement understanding to
22 executable PE design generation. The evaluator checks whether a generated
23 design is consistent with the task requirement, uses a suitable converter
24 structure, follows physically valid equations, satisfies component feasibility
25 and safety constraints, reports valid performance metrics, and supports
26 its design claims with calculation or simulation evidence. For a Generic
27 Two-Role MAS baseline on the 78-task suite, simulation-only evaluation
28 accepts 52.4% of candidates, whereas PE-Bench counts 23.5% as verifiable
29 task successes; for our PE-structured reference system, the corresponding
30 rates are 74.2% and 68.4%. This gap demonstrates the effectiveness of
31 domain-specific evaluation: PE-Bench distinguishes truly valid PE designs
32 by jointly assessing requirement alignment, converter-structure suitability,
33 physical consistency, component feasibility, safety constraints, performance
34 metrics, and evidence-supported design claims. The gap shows that many AI-
35 generated designs can appear executable while still lacking the engineering
36 consistency required for reliable PE design. Therefore, PE-Bench provides
37 a stricter, more faithful, and reusable evaluation framework for developing,
38 comparing, and improving agent-based automatic power-electronics design
39 systems.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

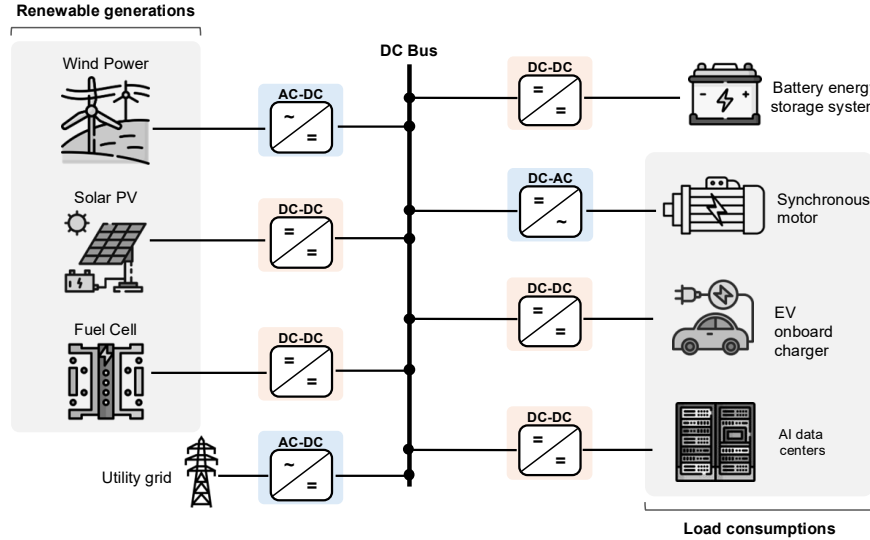


Figure 1: Power electronics as conversion infrastructure in a renewable energy system. Power electronic devices connect renewable energy sources to loads and enable critical applications in EVs, grids, data centers, and industrial drives.

1 Introduction

Power electronics (PE) provides the power-conversion hardware used in electric vehicles, renewable-energy interfaces, grid converters, industrial drives, server power supplies, and data-center power delivery. As electrification and high-performance computing expand, PE design has become an important engineering problem: an error in circuit-structure choice, device rating, magnetic sizing, or thermal margin can cause efficiency loss, overstress, component failure, or unsafe operation. Standard PE design references emphasize that topology choice, operating mode, magnetics, switch stress, passive-component ratings, and safety margin must be evaluated together [Mohan et al., 2003, Erickson and Maksimović, 2001, Pressman et al., 2009, Basso, 2014]. Recent energy reports show the scale of this demand. Global electricity demand grew around 3% in 2025. EV electricity demand may rise from about 180 TWh in 2024 to 780 TWh by 2030, and data-center electricity consumption is projected to reach around 945 TWh by 2030 in the base case [International Energy Agency, 2026, 2025b,a].

AI-assisted workflows are beginning to automate PE modeling, component selection, optimization, simulation setup, and design-report generation. Earlier neural-network and machine-learning work studied PE control, grid-facing PE systems, and converter optimization [Bose, 2007, Zhao et al., 2021, Abu-Rub et al., 2021, Rajamony et al., 2022]; more recent PE-GPT and physics-informed LLM-agent studies show that language models can assist PE workflows through equations, tools, and simulation interfaces [Lin et al., 2024, 2025, Liu et al., 2026]. A practical converter design requires the circuit structure, operating assumptions, equations, components, safety margins, simulation configuration, metrics, and final claims to agree. The evaluation question is therefore not only whether an AI system writes a readable report, but whether the generated PE design is internally consistent.

Simulation-backed engineering benchmarks are an important step beyond static answer matching. Broad LLM evaluations measure knowledge and reasoning [Hendrycks et al., 2021, Srivastava et al., 2023, Liang et al., 2022], while agent benchmarks test web, software, assistant, and ML-engineering workflows [Liu et al., 2024, Zhou et al., 2024, Jimenez et al., 2024, Mialon et al., 2024, Huang et al., 2024, Chan et al., 2025]. ENGDESIGN evaluates open-ended designs with simulation-backed functional verification [Guo et al., 2025], and closed-loop simulation benchmarks show the value of executable environments [Caesar et al., 2021]. Engineering-design benchmarks show that a task can have many valid solutions, so evaluation must accept different feasible candidates while still grounding success in executable evidence. PE adds a domain-specific need: a converter can run in simulation while requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints,

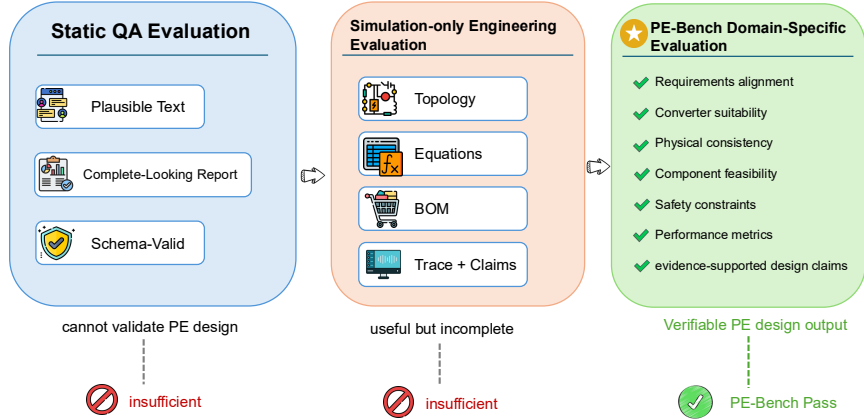


Figure 2: From simulation evidence to verifiable PE design. PE-BENCH validates task success when simulation evidence agrees with requirement alignment, physical consistency, component feasibility, safety constraints, metrics, claims, and review/escalation decisions.

performance metrics, or evidence-supported design claims remain unresolved. For these converter designs, simulation execution is useful evidence, and PE evaluation must connect that evidence to the full design claim.

We introduce PE-BENCH, a benchmark and evaluator for agent-based PE design systems. Each task contains a natural-language requirement, a machine-readable specification, and a feasible reference design. A candidate achieves verifiable task success when its requirement interpretation, converter structure, equations, bill of materials (BOM), component ratings, safety margins, simulation results, reported metrics, and review/escalation decision are mutually consistent. The review/escalation decision is a required candidate field indicating whether the agent signs off the design or escalates it for human engineering review; PE-BENCH automatically checks whether that decision is justified by the submitted evidence. Figure 2 summarizes this evaluation target.

This paper reports all main results on one unified 78-task benchmark covering Buck, Boost, Buck-Boost, Flyback, and three-phase DC-AC inverter tasks. The artifact records final runs, evaluator versions, simulation setup information, seed assignments, fault-injected validation, independently built valid designs, held-out evaluation, and data-leakage tests. For comparison, we also report our PE-structured multi-agent system, labeled STRONG BASELINE. PE-BENCH is not designed around STRONG BASELINE: tasks, reference designs, evaluator rules, and pass criteria are frozen independently of this reference system.

Contributions. First, we release PE-BENCH, a 78-task executable benchmark and evaluator for PE design consistency across five converter families; each task includes a requirement, structured specification, feasible reference design, and PE-specific success criteria rather than a single exact answer. Second, we validate the evaluator with fault-injected candidates, independently constructed valid designs, simulation-setup tests, held-out tasks, and leakage checks. Third, we evaluate a baseline set and demonstrate that PE-BENCH distinguishes simulation-only acceptance from verifiable task success by revealing hidden engineering inconsistencies in executable design candidates.

PE-BENCH is not an exact-match benchmark: reference designs are feasibility witnesses, and independently constructed alternatives are accepted. It is not a simulator-only benchmark: simulation evidence must agree with requirements, equations, components, ratings, metrics, and review/escalation decisions. It is not a product-certification benchmark: passing indicates consistency under the released criteria, while hardware validation and regulatory approval remain outside scope.

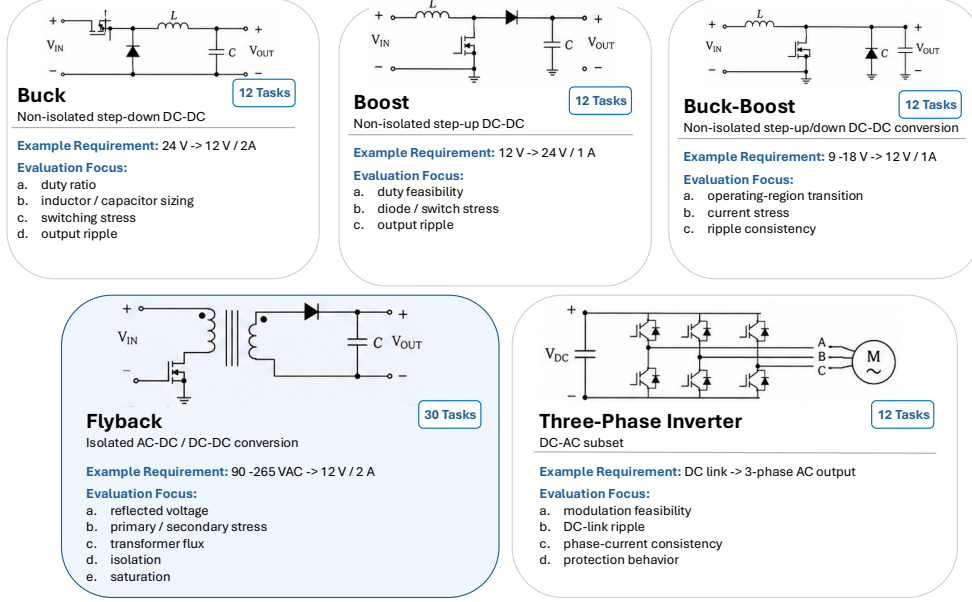


Figure 3: Unified 78-task converter coverage. PE-BENCH covers Buck, Boost, Buck-Boost, Flyback, and three-phase DC-AC inverter tasks with criteria for converter suitability, physical consistency, component feasibility, safety, supported metrics, and protection behavior.

2 Benchmark Design and Evaluation Target

2.1 Verifiable PE Design as the Evaluation Target

We evaluate a generated PE design by assessing whether its main design elements are mutually consistent. A candidate must maintain consistency among its requirement interpretation, converter structure, equations, BOM, component ratings, safety margins, simulation setup, reported metrics, and review/escalation decision. The converter structure must suit the isolation requirement, conversion ratio, power level, and operating mode; the equations must be physically consistent; the BOM must be feasible; and calculation or simulation evidence must support the reported metrics.

The preceding criteria make the PE-BENCH evaluation target explicit: the output is treated as a PE design evidence package. A candidate achieves verifiable task success when the required PE criteria are satisfied together. PE-BENCH identifies unresolved engineering inconsistencies even when a candidate follows the requested format or receives a high partial score. The unresolved criteria include requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints, performance metrics, and evidence-supported design claims.

Each task contains a natural-language requirement, machine-readable specification, feasible reference design, and run-record information. The specification records input range, output target, switching-frequency range, efficiency and ripple targets, cost proxy, safety constraints, difficulty tier, split, and converter family. The agent returns interpreted requirements, converter structure, equations, BOM, component ratings, safety margins, simulation setup, reported metrics, and review/escalation decision. Figure 3 summarizes the five-family task coverage; the selected families cover canonical non-isolated, isolated, and DC-AC converter settings [Mohan et al., 2003, Erickson and Maksimović, 2001, Pressman et al., 2009, Basso, 2014].

The benchmark release contains 78 executable tasks: 12 Buck, 12 Boost, 12 Buck-Boost, 30 Flyback, and 12 three-phase DC-AC inverter tasks. The main benchmark consists of 60 public-development tasks, six held-out tasks, and the 12-task DC-AC inverter subset; all are included in the main reported evaluation unless otherwise stated. PE-BENCH is intentionally a design-closure benchmark rather than a broad task-count benchmark: each task requires

a complete evidence package and multiple coupled checks. Because PE requirements can be satisfied by multiple component choices and operating points, reference designs serve as feasibility anchors rather than exact-match answers.

2.2 Evaluation criteria and metrics

PE-BENCH combines shared PE criteria with topology-specific physics criteria. Shared criteria cover required fields, requirement interpretation, component feasibility, safety margins, reported-value support, review/escalation decisions, and run-record completeness. Topology-specific criteria apply these checks to Buck, Boost, Buck-Boost, Flyback, and three-phase inverter tasks; Appendix D gives the full list.

The primary metric is verifiable task success rate (VTSR). VTSR is the fraction of task-seed evaluations that achieve PE-Bench verifiable task success:

$$\text{VTSR} = \frac{1}{|T||S|} \sum_{t \in T} \sum_{s \in S} \mathbf{1} \left[\bigwedge_{g \in G(t)} g(c_{t,s}) = 1 \right],$$

where T is the task set, S is the seed set, $G(t)$ is the required criterion set, and $c_{t,s}$ is the candidate output. A VTSR pass can correspond either to evidence-supported design sign-off or to a justified review/escalation decision when the submitted evidence makes safe sign-off unsupported. Partial score reports progress toward a full pass, while VTSR validates complete verifiable task success. Partial scores are diagnostic only and are not used to define leaderboard success. We also report Pass@ k , unsupported-performance-value rate, invalid/unsafe component-selection rate, simulator calls, retry regressions, and evaluator false-negative rate.

3 Benchmark Construction and Evaluator Validation

A benchmark for engineering agents must validate its evaluator as part of the contribution. PE-BENCH separates task construction from agent evaluation and follows documentation, metadata, and versioning practices for dataset and model reporting [Geburu et al., 2021, Mitchell et al., 2019, Akhtar et al., 2024]. For each converter family, we create task seeds, convert them into machine-readable specifications, pair them with feasible reference designs, construct fault-injected candidates, and freeze the evaluator version. Run records store simulation setup, method label, seed, fallback behavior, evaluator version, and checksum. Figure 4 summarizes the pipeline.

Reference designs show task feasibility and help set the expected operating range; they are not exact-match gold answers. Difficulty tiers are assigned from reference-design margins and baseline outcomes. Easy tasks have wide margins; medium and hard tasks tighten efficiency, ripple, or duty-cycle requirements; boundary and stress tasks are close to operating transitions, component limits, or review/escalation cases.

Table 1 summarizes the validation suite. The validation suite is separate from the leaderboard and demonstrates that PE-BENCH measures domain-specific design consistency beyond reference matching or format compliance.

The fault-injected validation introduces realistic engineering inconsistencies into plausible candidates, covering requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints, performance metrics, and review/escalation behavior. The independent-valid-design validation includes 100 alternatives across all five families: 94 are accepted. Among the six non-accepted alternatives, four are routed to justified review/escalation categories and two are recorded as evaluator false negatives. These validations rule out three trivial interpretations of PE-BENCH: format checking is insufficient because malformed-output tests are separated from fault-injected engineering inconsistencies; reference matching is insufficient because 94 independently constructed non-reference feasible designs are accepted; and simulator-only scoring is insufficient because injected stress, BOM, and reported-metric faults are detected even when candidates remain plausible or executable.

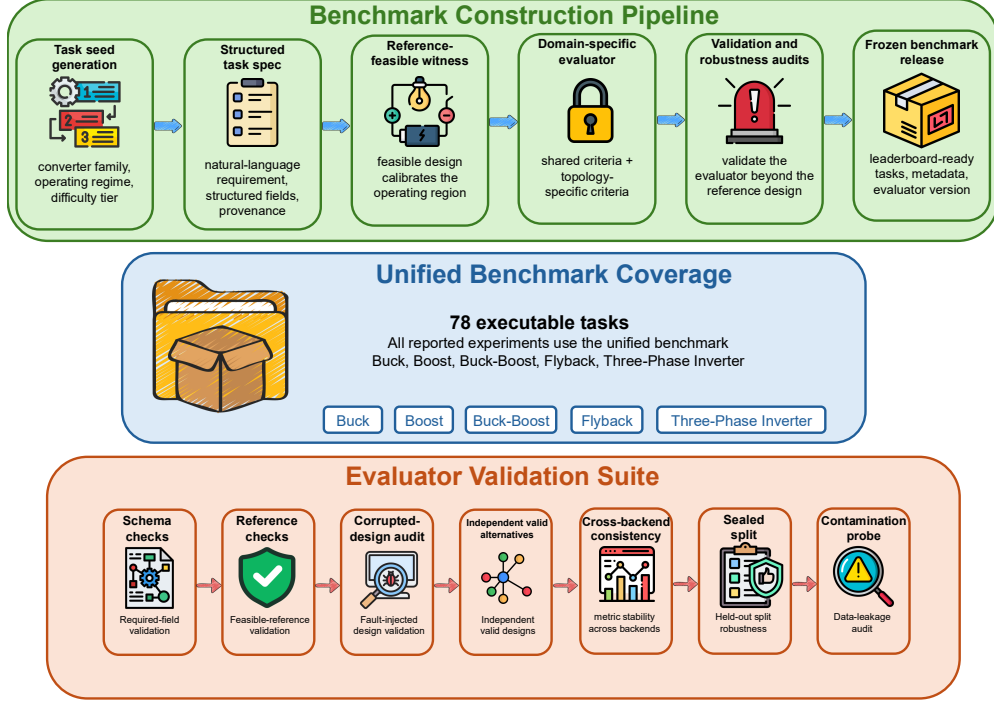


Figure 4: Benchmark construction and evaluator validation pipeline. Tasks are generated, paired with feasible reference designs, validated with fault-injected and independently built valid designs, and frozen for release. The suite demonstrates that PE-BENCH measures design consistency beyond reference matching.

4 Experimental Protocol

We evaluate a baseline set that separates output-format control, self-refinement, tool access, retry, role decomposition, PE-domain prompting, and PE-structured agent design [Wei et al., 2022, Yao et al., 2023a, Madaan et al., 2023, Shinn et al., 2023, Yao et al., 2023b, Schick et al., 2023, Wu et al., 2024a, Li et al., 2023, Hong et al., 2024, Qian et al., 2024]. The baseline set includes LLM-only, Structured-output, Text Self-Refine, LLM+Tools, Single-Agent+Retry, Generic Two-Role MAS, PE-GPT-style prompting [Lin et al., 2024, 2025], and STRONG BASELINE. All systems receive the same task cards, catalog slice, output schema, seed assignment, attempt budget, retry cap, simulator-access policy, and evaluator interface; no task-specific prompt tuning is performed on held-out tasks. STRONG BASELINE is our PE-structured multi-agent system and is used only as a reference system for benchmark comparison; it is not part of the PE-BENCH task suite, evaluator, or pass criterion.

The main leaderboard covers all 78 tasks with three seeds per method and reports VTSR, partial score, Pass@3, unsupported-performance-value rate, invalid/unsafe component-selection rate, and average simulator calls. Unless otherwise stated, all leaderboard and simulation-gap results are computed over 78 tasks $\times$ three seeds for each method. Additional experiments compare simulation-only acceptance with VTSR, ablate PE-specific checking modules inside STRONG BASELINE while keeping the final evaluator unchanged, compare model classes, evaluate held-out tasks, stratify by difficulty, and sweep attempt budgets.

5 Results and analysis

The experiments test whether PE-BENCH separates methods, reveals the simulation-to-VTSR gap, identifies the effect of each PE-specific design-checking module, and gives stable results. Numeric tables and stability analyses are in Appendix G.

Table 1: Compact evaluator validation summary. The suite tests fault detection, feasible design diversity, held-out stability, simulator-setup stability, and leakage risk.

Validation focus	Cases	Main result
Task and run-record integrity	78 tasks; all runs	Required fields, feasible references, simulation setup, seeds, evaluator version, fallback behavior, and checksums are recorded.
Fault and malformed-output detection	500+ malformed; 800 faults	Parse validation identifies malformed outputs; PE-BENCH identifies 99.2% of injected inconsistencies with 0.0% unsafe false acceptance.
Non-reference valid designs	100 alternatives	94 independently constructed feasible designs are accepted; among six non-accepted alternatives, four are routed to justified review/escalation categories and two are evaluator false negatives.
Stability and leakage checks	6 held-out tasks / 24 records; 60 setup tests; 1000 slots; 8 synchronous-buck tasks	Held-out ordering is stable, simulator-setup agreement is 95.0%, no leakage flags are found, and the synchronous-buck mini-track is excluded from the 78-task leaderboard while supporting extensibility.

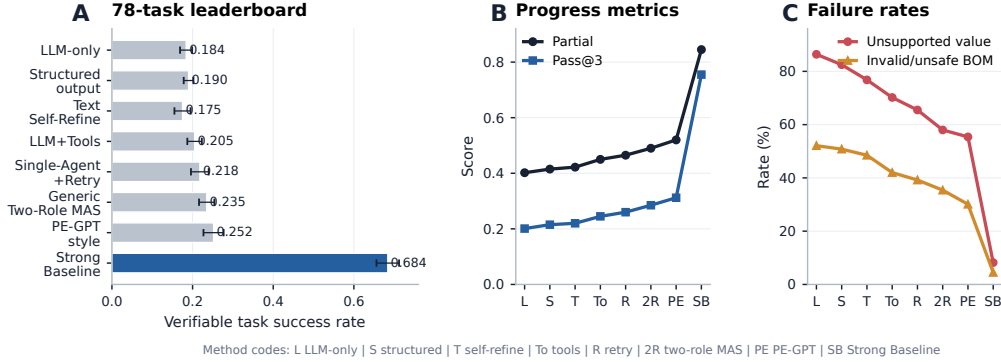


Figure 5: Main 78-task results. All method-level results use 78 tasks $\times$ three seeds unless otherwise stated. (A) VTSR across three seeds. (B) Diagnostic partial score and Pass@3. (C) Unsupported-value and invalid/unsafe component-selection rates.

5.1 Main result: PE-Bench identifies verifiable PE design success

The 78-task leaderboard shows that PE-BENCH separates PE-structured agent design from generic prompting and tool-use baselines. Figure 5 summarizes the comparison. STRONG BASELINE reaches 0.684 ± 0.028 VTSR, while baselines range from 0.175 to 0.252. STRONG BASELINE also obtains 0.845 diagnostic partial score and 0.755 Pass@3. The VTSR and Pass@3 results show that PE-BENCH identifies unresolved inconsistencies in requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints, performance metrics, and evidence-supported design claims.

The comparison also distinguishes domain-specific PE evaluation from tool access alone: LLM+Tools, Single-Agent+Retry, and Generic Two-Role MAS can call tools or use feedback, yet their outputs still show a large gap to verifiable task success. Many candidate reports look complete, but PE-BENCH reveals gaps in requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints, performance metrics, or evidence-supported design claims.

5.2 The simulation-to-VTSR gap demonstrates the value of domain-specific evaluation

Figure 6 demonstrates the effectiveness of domain-specific evaluation by comparing simulation-only acceptance with PE-BENCH verifiable task success over the same 78-task $\times$ three-seed candidate pool used in Figure 5. Baselines often reach simulator execution or pass simulation-

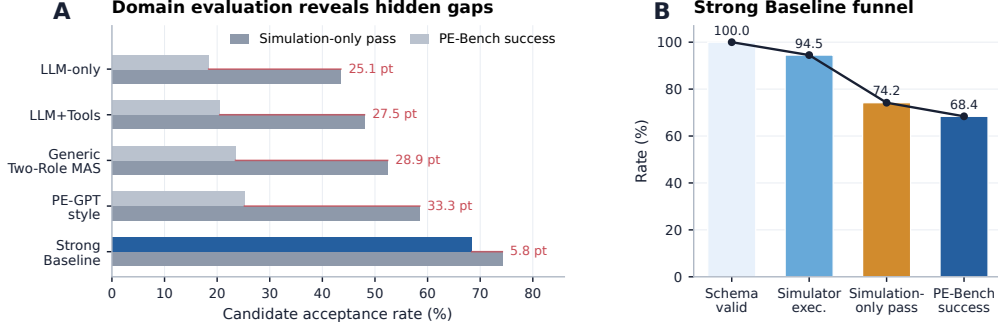


Figure 6: The simulation-to-VTSR gap reveals hidden PE design inconsistencies. Panel A reports method-level gaps using the same 78-task $\times$ three-seed candidate pool as Figure 5; Panel B reports the stage-wise funnel for STRONG BASELINE only. Rigorous domain-specific evaluation identifies unresolved requirement-alignment, converter-suitability, physical-consistency, component-feasibility, safety, reported-metric support, and review/escalation issues.

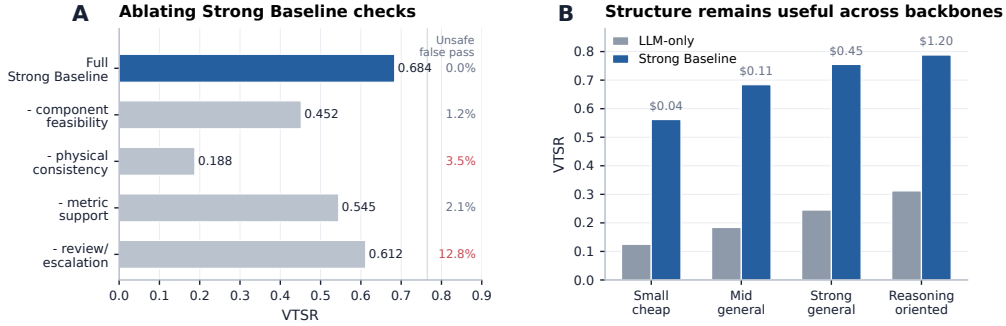


Figure 7: Ablating PE-specific design-checking modules and comparing model classes. All ablated systems are evaluated by the full PE-BENCH evaluator. Component-feasibility, physical-consistency, reported-metric, and review/escalation modules each protect a different part of verifiable task success. Dollar labels denote estimated cost per task.

level evaluation. PE-BENCH identifies fewer candidates as verifiable task successes because it reveals hidden engineering inconsistencies beyond simulator execution. LLM-only shows a 25.1 percentage-point gap; PE-GPT-STYLE shows the largest gap at 33.3 points. Figure 6B reports the stage-wise funnel for STRONG BASELINE only.

The simulation-to-VTSR gaps reflect PE design evidence, not formatting alone. STRONG BASELINE narrows the gap to 5.8 points, mostly in cases requiring escalation rather than unsupported sign-off. The 5.8-point gap supports the central claim: simulation is useful evidence, and verifiable PE design success requires domain-specific evaluation.

Appendix Table 4 gives two representative candidates behind this gap: one simulator-pass design fails because stress and BOM evidence are unsupported, while one non-reference design passes because equations, ratings, margins, and metrics are supported.

5.3 PE-specific checking modules remain necessary across model classes

Figure 7 ablates PE-specific design-checking modules inside STRONG BASELINE while keeping the final PE-BENCH evaluator unchanged. Removing component-feasibility checking lowers final VTSR from 0.684 to 0.452 and raises invalid/unsafe component-selection rate from 4.5% to 46.8%. Removing physical-consistency checking gives 0.188 VTSR, close to LLM-only performance. Removing reported-metric checking gives 0.545 VTSR but raises unsupported-performance-value rate to 68.4%, revealing a hidden inconsistency that simulation-only evaluation misses. Removing the review/escalation rule gives 0.612 VTSR but raises unsafe false-acceptance rate to 12.8%, showing that escalation behavior is part of verifiable engineering success.

Stronger model classes help, and the need for domain-specific evaluation remains: across four anonymized model classes, LLM-only VTSR rises from 0.125 to 0.312, while STRONG BASELINE rises from 0.562 to 0.788.

5.4 Evaluator testing and stability

Table 1 reports compact evaluator-validation evidence; Appendix E and Appendix G give fault-injected categories, valid-design tests, held-out tasks, difficulty tiers, retry-budget tests, and evaluator-configuration robustness results. The experiments preserve method ordering across setup changes, supporting VTSR as a verifiable-success rate beyond reference matching.

6 Related work, limitations, and conclusion

Related work. Classical PE texts define the coupled design constraints behind converter topology, operating mode, magnetics, component stress, passive sizing, and simulation-backed verification [Mohan et al., 2003, Erickson and Maksimović, 2001, Pressman et al., 2009, Basso, 2014]. AI for PE has studied neural-network-based control, diagnosis, optimization, and grid-facing PE applications [Bose, 2007, Zhao et al., 2021, Abu-Rub et al., 2021, Rajamony et al., 2022]. PE-GPT-style and physics-informed LLM-agent work show early language-model-assisted PE workflows [Lin et al., 2024, 2025, Liu et al., 2026], but these studies do not define a reusable benchmark for requirement alignment, converter suitability, component feasibility, safety constraints, and evidence-supported design claims.

Engineering-design benchmarks move LLM evaluation from textbook answers toward executable design tasks: ENGDESIGN evaluates multi-domain engineering designs with simulation-backed verification [Guo et al., 2025], ControlAgent studies agent-based control-system design with domain expertise [Guo et al., 2024], and FDM-Bench evaluates additive-manufacturing tasks [Eslaminia et al., 2024]. General LLM and agent evaluations cover knowledge, reasoning, web tasks, software repair, assistant tasks, and ML engineering [Hendrycks et al., 2021, Srivastava et al., 2023, Liang et al., 2022, Liu et al., 2024, Zhou et al., 2024, Jimenez et al., 2024, Mialon et al., 2024, Huang et al., 2024, Chan et al., 2025].

Agent-building work studies reasoning traces, tool use, iterative self-refinement, role-based collaboration, and multi-agent software workflows [Wei et al., 2022, Yao et al., 2023b, Schick et al., 2023, Madaan et al., 2023, Shinn et al., 2023, Yao et al., 2023a, Wu et al., 2024a, Li et al., 2023, Hong et al., 2024, Qian et al., 2024]. Agent methods motivate stronger PE design agents, including our PE-structured multi-agent reference system STRONG BASELINE, but agent methods are separate from the PE-BENCH task suite and evaluator. PE-BENCH builds on executable-design benchmarks and focuses on a PE-specific target: a design can be simulator-executable while equations, converter choice, BOM ratings, reported metrics, or sign-off decisions remain inconsistent. Appendix Table 3 summarizes PE-BENCH’s boundary with general LLM QA, agent benchmarks, ENGDESIGN, and PE-GPT-style workflow demonstrations. The PE-BENCH task cards, run records, evaluator cards, and release metadata follow dataset/model documentation practices and benchmark-robustness discussions [Geburu et al., 2021, Mitchell et al., 2019, Akhtar et al., 2024, Bhardwaj et al., 2024, Wu et al., 2024b, Haimen et al., 2024].

Limitations. PE-BENCH prioritizes evaluation depth over domain breadth. The current release covers Buck, Boost, Buck-Boost, Flyback, a three-phase DC-AC inverter subset inside the 78-task benchmark, and a small synchronous-buck mini-track outside the 78-task leaderboard. Future tracks should include resonant converters, power-factor-correction (PFC) converters, thermal layout, EMI/EMC, and compliance tasks. Passing a benchmark task means that a candidate satisfies PE-Bench criteria; product build, sale, and certification still require engineering review and hardware testing.

Conclusion. We introduced PE-BENCH, a 78-task benchmark for evaluating whether AI agents produce PE designs with consistent requirements, equations, components, safety margins, simulation results, and final reports. The results reveal large gaps between simulation-only acceptance and verifiable task success, demonstrating the effectiveness of domain-specific evaluation. PE-BENCH uses PE as a compact engineering testbed where design mistakes have clear technical consequences. PE-BENCH distinguishes executable-looking outputs from evidence-supported PE designs and enables clearer development, comparison, and debugging of PE design agents.

References

- Omar H. Abu-Rub, Amin Y. Fard, Muhammad Farooq Umar, Mohsen Hosseinzadehtaher, and Mohammad B. Shadmans. Towards intelligent power electronics-dominated grid via machine learning techniques. *IEEE Power Electronics Magazine*, 8(1):28–38, 2021. doi: 10.1109/MPPEL.2020.3047506.
- Mubashara Akhtar et al. Croissant: A metadata format for ML-ready datasets. In *Advances in Neural Information Processing Systems*, 2024. doi: 10.52202/079017-2610.
- Christophe P. Basso. *Switch-Mode Power Supplies: SPICE Simulations and Practical Designs*. McGraw-Hill, 2014.
- Eshta Bhardwaj, Harshit Gujral, Siyi Wu, Ciara Zogheib, Tegan Maharaj, and Christoph Becker. The state of data curation at neurips: An assessment of dataset development practices in the datasets and benchmarks track. In *Advances in Neural Information Processing Systems*, 2024.
- Bimal K. Bose. Neural network applications in power electronics and motor drives—an introduction and perspective. *IEEE Transactions on Industrial Electronics*, 54(1):14–33, 2007. doi: 10.1109/TIE.2006.888683.
- Holger Caesar, Juraj Kabzan, Kok Seang Tan, Eric Wolff Fong, et al. nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.
- Robert W. Erickson and Dragan Maksimović. *Fundamentals of Power Electronics*. Springer, 2001.
- Ahmadreza Eslaminia, Adrian Jackson, Beitong Tian, Avi Stern, Hallie Gordon, Rajiv Malhotra, Klara Nahrstedt, and Chenhui Shao. FDM-Bench: A comprehensive benchmark for evaluating large language models in additive manufacturing tasks. *arXiv preprint arXiv:2412.09819*, 2024.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- Xingang Guo, Darioush Keivan, Usman Syed, Lianhui Qin, Huan Zhang, Geir Dullerud, Peter Seiler, and Bin Hu. ControlAgent: Automating control system design via novel integration of LLM agents and domain expertise. *arXiv preprint arXiv:2410.19811*, 2024.
- Xingang Guo, Yaxin Li, Xiangyi Kong, Yilan Jiang, Xiayu Zhao, Zhihua Gong, Yufan Zhang, et al. Toward engineering agi: Benchmarking the engineering design capabilities of llms. In *Advances in Neural Information Processing Systems*, 2025.
- Jacob Haimès, Cenny Wenner, Kunvar Thaman, Vassil Tashev, Clement Neo, Esben Kran, and Jason Schreiber. Benchmark inflation: Revealing llm performance gaps using retro-holdouts. *arXiv preprint arXiv:2410.09247*, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

- 352 Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating lan-
 353 guage agents on machine learning experimentation. In *Proceedings of the 41st International*
 354 *Conference on Machine Learning*, pages 20271–20309, 2024.
- 355 International Energy Agency. Energy and AI. IEA, Paris, [https://www.iea.org/reports/](https://www.iea.org/reports/energy-and-ai)
 356 [energy-and-ai](https://www.iea.org/reports/energy-and-ai), 2025a. Licence: CC BY 4.0.
- 357 International Energy Agency. Global EV Outlook 2025. IEA, Paris, [https://www.iea.org/](https://www.iea.org/reports/global-ev-outlook-2025)
 358 [reports/global-ev-outlook-2025](https://www.iea.org/reports/global-ev-outlook-2025), 2025b. Licence: CC BY 4.0.
- 359 International Energy Agency. Global Energy Review 2026. IEA, Paris, [https://www.iea.](https://www.iea.org/reports/global-energy-review-2026)
 360 [org/reports/global-energy-review-2026](https://www.iea.org/reports/global-energy-review-2026), 2026. Licence: CC BY 4.0.
- 361 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
 362 Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues?
 363 In *International Conference on Learning Representations*, 2024.
- 364 Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard
 365 Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model
 366 society. In *Advances in Neural Information Processing Systems*, pages 51991–52008, 2023.
- 367 Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga,
 368 Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of
 369 language models. *arXiv preprint arXiv:2211.09110*, 2022.
- 370 Fanfan Lin, Junhua Liu, Xinze Li, Shuai Zhao, Bohui Zhao, Xinyuan Liao, Hao Ma, and Xin
 371 Zhang. PE-GPT: A physics-informed interactive large language model for power converter
 372 modulation design. In *2024 IEEE Energy Conversion Congress and Exposition (ECCE)*,
 373 pages 1744–1747, 2024. doi: 10.1109/ECCE55643.2024.10861072.
- 374 Fanfan Lin, Xinze Li, Weihao Lei, Juan J. Rodriguez-Andina, Josep M. Guerrero, Changyun
 375 Wen, Xin Zhang, and Hao Ma. PE-GPT: A new paradigm for power electronics design.
 376 *IEEE Transactions on Industrial Electronics*, 72(4):3778–3791, 2025. doi: 10.1109/TIE.
 377 2024.3454408.
- 378 Junhua Liu, Fanfan Lin, Xinze Li, Shuai Zhao, and Kwan Hui Lim. Physics-informed au-
 379 tonomous LLM agents for explainable power electronics modulation design. In *Proceedings*
 380 *of the 40th Annual AAAI Conference on Artificial Intelligence*, pages 40063–40072. AAAI
 381 Press, 2026. doi: 10.1609/aaai.v40i47.41441.
- 382 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang
 383 Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In
 384 *International Conference on Learning Representations*, 2024.
- 385 Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe,
 386 Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative
 387 refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- 388 Alfonso Martinez. OpenMagnetics: Online toolbox for designing and simulating magnetics.
 389 *IEEE Power Electronics Magazine*, 11(3):37–50, 2024. doi: 10.1109/MPEL.2024.3440706.
- 390 Gregoire Mialon, Clementine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom.
 391 GAIA: A benchmark for general AI assistants. In *International Conference on Learning*
 392 *Representations*, 2024.
- 393 Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben
 394 Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards
 395 for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and*
 396 *Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- 397 Ned Mohan, Tore M. Undeland, and William P. Robbins. *Power Electronics: Converters,*
 398 *Applications, and Design*. Wiley, 2003.

- 399 Plexim GmbH. PLECS: The simulation platform for power electronic systems. <https://www.plexim.com/products/plecs>, 2026. Power-electronics simulation software docu-
400 mentation.
401
- 402 Abraham I. Pressman, Keith Billings, and Taylor Morey. *Switching Power Supply Design*.
403 McGraw-Hill, 2009.
- 404 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize
405 Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun.
406 ChatDev: Communicative agents for software development. In *Proceedings of the 62nd*
407 *Annual Meeting of the Association for Computational Linguistics*, pages 15174–15186,
408 2024. doi: 10.18653/v1/2024.acl-long.810.
- 409 Rajesh Rajamony, Sheng Wang, Gerardo Calderon-Lopez, Ingo Ludtke, and Wenlong Ming.
410 Artificial neural networks-based multi-objective design methodology for wide-bandgap
411 power electronics converters. *IEEE Open Journal of Power Electronics*, 3:599–610, 2022.
412 doi: 10.1109/OJPEL.2022.3204630.
- 413 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke
414 Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can
415 teach themselves to use tools. In *Advances in Neural Information Processing Systems*,
416 2023.
- 417 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan,
418 and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv*
419 *preprint arXiv:2303.11366*, 2023.
- 420 Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid,
421 Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adria Garriga-Alonso, et al.
422 Beyond the imitation game: Quantifying and extrapolating the capabilities of language
423 models. *Transactions on Machine Learning Research*, 2023.
- 424 Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi,
425 Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language
426 models. In *Advances in Neural Information Processing Systems*, 2022.
- 427 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang,
428 Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM
429 applications via multi-agent conversation. In *Conference on Language Modeling*, 2024a.
- 430 Yiwei Wu, Leah Ajmani, Shayne Longpre, and Hanlin Li. A systematic review of neurips
431 dataset management practices. *arXiv preprint arXiv:2411.00266*, 2024b.
- 432 Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and
433 Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language
434 models. *arXiv preprint arXiv:2305.10601*, 2023a.
- 435 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and
436 Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International*
437 *Conference on Learning Representations*, 2023b.
- 438 Shuai Zhao, Frede Blaabjerg, and Huai Wang. An overview of artificial intelligence applica-
439 tions for power electronics. *IEEE Transactions on Power Electronics*, 36(4):4633–4658,
440 2021. doi: 10.1109/TPEL.2020.3024914.
- 441 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi
442 Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. WebArena: A realistic web
443 environment for building autonomous agents. In *International Conference on Learning*
444 *Representations*, 2024.

Table 2: Reviewer questions answered by the appendix.

Reviewer question	Where it is answered
What exactly does PE-BENCH evaluate?	Appendix B defines the evaluation target, scope, non-use cases, and task coverage.
Are the tasks feasible without forcing one gold design?	Appendix C describes task cards and feasible reference designs; the reference is a feasible design, not an exact-match solution.
What does a candidate have to submit?	Appendix D specifies the prompt payload, candidate design output, result format, evaluation criteria, and failure tags.
Why should the evaluator be trusted?	Appendix E reports fault injection, independent valid-design validation, simulation-setup consistency, held-out split evaluation, and data-leakage tests.
Were baselines compared fairly?	Appendix F records the baseline ladder, shared task interface, attempt budgets, tool access, and runtime/cost accounting.
What additional results support the main conclusions?	Appendix G gives compact simulation-only versus VTSR analysis, ablations, backbone comparison, attempt-budget analysis, held-out-split results, and extensibility results.
Can the release be re-run and safely interpreted?	Appendix H gives the release layout, quickstart, run-record checklist, and non-use statement.

A Appendix Roadmap: A Reviewer-Oriented Guide

The appendix is organized as a response to the main questions a reviewer may ask about a new engineering-evaluation benchmark. The goal is not to duplicate the code repository or list every task row in the manuscript, but to make the evaluation target, task construction, evaluator rules, validation support, experimental protocol, and release verifiable.

B Benchmark Card and Scope

Evaluation target. PE-BENCH evaluates whether AI-assisted PE design outputs achieve verifiable task success under a domain-specific PE evaluation protocol. A candidate is counted as verifiable when its requirement interpretation, converter choice, topology/mode, governing equations, BOM, component ratings, safety margins, simulator- or formula-backed metrics, reported performance values, and review/escalation decision are mutually consistent. The evaluator distinguishes evidence-supported PE designs from outputs that are well written, format-valid, or simulator-executable but lack complete design evidence.

Why PE is a useful testbed. PE design reveals an evaluation gap that general engineering benchmarks can miss: a converter can execute in simulation while still lacking requirement alignment, converter suitability, physical consistency, component feasibility, safety constraints, or evidence-supported performance claims. PE is therefore a compact engineering setting for testing whether agents keep design representations consistent.

Scope. The unified benchmark contains 78 executable tasks across five families: Buck, Boost, Buck-Boost, Flyback, and a three-phase inverter DC-AC subset. The task set favors depth over breadth: it covers representative evaluation criteria for converter-design consistency, while future tracks can add more PE topologies, layout issues, thermal design, EMI/EMC requirements, regulatory standards, and hardware validation workflows.

Intended and non-intended use. The benchmark is intended for evaluating and debugging AI systems that generate PE design outputs. It is not intended for production sign-off, automated procurement, certified safety review, medical or grid-code approval, or replacement of qualified PE engineers.

Table 3: Boundary with related evaluations. PE-BENCH focuses on executable PE design closure rather than general task completion or PE workflow demonstration.

Work / benchmark	Executable	Domain physics	BOM/rating	Non-reference designs	Sign-off / escalation
General LLM QA	No	No	No	No	No
AgentBench / GAIA / WebArena	Partial	No	No	Task-dependent	No
ENGDESIGN	Yes	Broad	Limited	Yes	Limited
PE-GPT-style work	Workflow	PE-specific	Not bench-marked	Not bench-marked	No reusable evaluator
PE-Bench	Yes	Yes	Yes	Yes	Yes

Table 4: Two examples behind the simulation-to-VTSR gap. The examples illustrate why PE-BENCH is neither simulator-only scoring nor exact reference matching.

Candidate	Simulation-only	Reference match	PE-BENCH	Reason
Simulator-pass near miss	Pass	No	Fail	Reported metrics were executable, but switch stress and BOM ratings were not supported by the submitted evidence.
Non-reference valid design	Pass	No	Pass	Component choices and switching frequency differed from the reference, but equations, ratings, margins, and metrics were supported.

C Task Construction and Feasibility Anchors

C.1 Construction pipeline

Each task begins with a converter family, operating regime, difficulty tier, and split assignment. The task seed is converted into a machine-readable specification and paired with a feasible reference design. The reference design demonstrates task feasibility and helps set the expected operating range; it serves as a feasibility anchor, not a unique gold answer. The task and reference design are then evaluated by shared and converter-specific criteria, validated with fault-injected candidates and independent valid alternatives, and stored with evaluator versions and run records.

C.2 Feasible reference designs as anchors

Multiple PE designs can satisfy the same requirement. The reference design therefore plays three roles: (i) demonstrating that the task is feasible, (ii) calibrating expected operating ranges and inconsistency points, and (iii) confirming that the evaluator recognizes a complete, safe, internally consistent candidate. The evaluator allows candidates to use different component identities, inductance values, switching frequencies, or cost proxies when the submitted evidence supports the design. The independent valid-design study in Appendix E directly validates this non-reference-design setting.

C.3 Representative task cards

The complete inventory is released as JSON/CSV with the artifact. Table 8 shows representative task cards to make the task distribution inspectable without turning the appendix into a data dump.

Table 5: Unified task coverage used in the main manuscript. The release stores the full machine-readable inventory; the appendix keeps a compact coverage table and representative task cards.

Topology	#	Easy	Med.	Hard	Bnd.	Stress	Pub-dev	Held-out	DC-AC
Flyback	30	6	10	6	4	4	24	6	0
Buck	12	4	4	2	1	1	12	0	0
Boost	12	4	4	2	1	1	12	0	0
Buck-Boost	12	4	4	2	1	1	12	0	0
Three-phase inverter	12	3	4	3	0	2	0	0	12
Total	78	21	26	15	7	9	60	6	12

Table 6: Task-card fields. The artifact stores these fields for each task; the manuscript shows representative examples and the released artifact stores the full inventory.

Field	Purpose
<code>task_id</code>	Immutable identifier used in result run records and figure/table reproduction scripts.
<code>natural_language_spec</code>	Reader-facing design request presented to the agent.
<code>structured_spec</code>	Machine-readable input range, output target, switching-frequency range, efficiency/ripple targets, safety constraints, preferences, and catalog version.
<code>difficulty_tier,</code> <code>split</code>	Difficulty and public, held-out, and DC-AC subset run records used for stratified analysis and data-leakage tests.
<code>reference_design</code>	Feasible anchor containing topology parameters, selected components, expected metrics, and a cost proxy.
<code>evaluation_rubric</code>	Criterion weights used for partial-score analysis; VTSR remains a binary verifiable-success metric.
<code>known_failure_modes</code>	Expected PE inconsistency tags used for interpretation and fault-injection design.

493 D Evaluator Interface and Evaluation Criteria

494 D.1 Prompt and candidate interface

495 Agents receive a structured prompt payload with the task card, a bounded catalog slice,
496 output instructions, and a response format. Candidates may include free-form rationale, and
497 the evaluator uses the machine-readable fields for scoring and run-record validation.

498 D.2 Result format and pass criterion

499 The evaluator emits both a binary verifiable-success decision and intermediate scores. VTSR
500 is an evidence-based verifiable-success metric: a candidate is counted as successful when the
501 submitted design evidence supports requirement alignment, physical consistency, component
502 feasibility, safety constraints, reported performance metrics, and the review/escalation
503 decision. Partial scores describe intermediate progress, while VTSR validates complete
504 verifiable task success.

505 D.3 Evaluation criteria by converter family

506 D.4 Engineering-inconsistency taxonomy

507 E Evaluator Validation and Generality

508 **Validation goal.** A benchmark for engineering agents must validate its evaluator as part
509 of the contribution. The validation suite answers three reviewer concerns: (i) whether the
510 evaluator identifies hidden engineering inconsistencies, (ii) whether the evaluator accepts
511 feasible designs that differ from the reference design, and (iii) whether evaluator decisions
512 remain stable when simulator setup, task split, or leakage conditions change.

Table 7: Feasible-reference fields by topology family.

Family	Feasibility-anchor fields
Flyback	topology; primary-to-secondary turns ratio; magnetizing inductance; switching frequency; maximum duty cycle; primary peak current; selected controller, MOSFET, diode, output capacitor, and core; expected efficiency, ripple, MOSFET stress, diode reverse stress, flux density, startup success; cost proxy.
Buck / Boost / Buck-Boost	topology; nominal duty cycle; inductance; output capacitance; switching frequency; inductor ripple current; switch peak current; selected controller, switch, diode, output capacitor, and inductor; expected efficiency, ripple, voltage stress, inductor peak current, startup success; cost proxy.
Three-phase inverter	topology; DC-link voltage; AC voltage or motor/load operating point; modulation strategy; switching frequency; phase-current estimate; device and gate-driver selections; DC-link capacitor rating; current-quality or traction-load metric; expected efficiency, stress margin, protection status, and cost proxy.

Table 8: Representative task cards. The full task bank is machine-readable in the artifact.

Task ID	Family	Tier	Requirement sketch and evaluation focus
easy_acdc_12v1a	Flyback	Easy	90–265 VAC to 12 V / 1 A isolated output; evaluates turns ratio, switch/diode stress, ripple, startup, and isolation consistency.
boundary_dcdc_12v3a_narrow_duty	Flyback	Bound.	36–60 VDC to 12 V / 3 A with tight duty and stress margins; evaluates whether a candidate escalates unclear cases or supports conservative margins.
topology_buck_hard_12v5a	Buck	Hard	42–58 VDC to 12 V / 5 A; evaluates duty feasibility, inductor/capacitor sizing, current stress, and reported-value support.
topology_boost_stress_36v1p5a	Boost	Stress	5–8 VDC to 36 V / 1.5 A; evaluates high conversion-ratio stress, diode/switch ratings, ripple, and efficiency values.
inverter_3ph_hard_grid_800v_100kw	Inverter	Hard	1500 V DC-link to 800 V AC rms / 100 kW grid-connected output; evaluates modulation feasibility, phase current, DC-link ripple, device stress, and protection behavior.

Final 78-task accounting. All required-field and feasible-reference counts in this version refer to the finalized 78-task benchmark. An earlier core-only validation run used a 66-task subset; that accounting is superseded here by the final five-family rerun. The synchronous-buck mini-track is reported as an additional extensibility validation and is not included in the 78-task leaderboard.

E.1 Fault-injected design validation

Engineering inconsistencies are deliberately introduced into otherwise plausible candidates. The fault-injection test evaluates whether PE-BENCH identifies hidden engineering inconsistencies that can remain inside plausible AI-generated PE reports. The injected cases cover component feasibility, physical consistency, converter suitability, safety constraints, and evidence-supported performance metrics.

E.2 Independent valid designs

The independent valid-design validation evaluates whether the evaluator accepts non-reference but feasible designs. The expanded validation includes 100 alternatives across all five topology

Table 9: Candidate design output format.

Group	Required fields
Shared fields	<code>task_id</code> ; design rationale; interpreted requirements; selected topology; BOM; final reported performance values; simulator configuration; review/escalation decision; complete design output.
Flyback theory	turns ratio; magnetizing inductance; switching frequency; maximum duty; primary peak current; transformer/core quantities.
Non-isolated theory	nominal duty cycle; inductance; capacitance; switching frequency; inductor ripple current; switch peak current.
Three-phase inverter theory	DC-link voltage; AC rms voltage or motor-load operating point; modulation index; switching frequency; phase-current estimate; dead time; current-quality or traction-load metric.
BOM/component validation	Category-part pairs for controller, switch/MOSFET, diode, output/DC-link capacitor, magnetic component, gate driver where applicable, and supporting parts.
Reported values and escalation	Efficiency, ripple/current-quality, stress margins, startup/protection status, cost proxy, and the evidence supporting either design sign-off or escalation for engineering review.

Table 10: Evaluator outputs.

Field	Meaning
<code>pass_fail / vtsr_pass</code>	Binary verifiable-task-success decision; true when all required PE evaluation criteria are satisfied.
<code>score_total, sub_scores</code>	Diagnostic partial scores for specification alignment, theory consistency, component-database validation, safety margins, simulation/formula metrics, reported-value consistency, and review/escalation.
<code>constraint_violations</code>	Structured list of unresolved constraints and missing fields.
<code>simulation_metrics</code>	Simulation-setup or formula-derived metrics used for reported-value validation.
<code>failure_tags / failure_groups</code>	Fixed taxonomy for grouping engineering inconsistencies.
<code>execution_log, runtime_stats</code>	Simulation-setup mode, fallback behavior, warnings, simulator calls, runtime, evaluator version, and run-record checksum.

families. Candidates that are not accepted on the first pass are categorized in the review log so that incomplete evidence, true unsafe margins, and evaluator false negatives are separated.

E.3 Simulation setup, held-out split, and data-leakage validation

To test whether PE-BENCH conclusions depend on a single evaluator configuration, we re-evaluate the same frozen candidate outputs under small changes to safety margins, reported-metric tolerances, and component-catalog availability. This check covers 1170 evaluations from 78 tasks, three seeds, and five representative methods: LLM-only, LLM+Tools, Generic Two-Role MAS, PE-GPT-STYLE, and STRONG BASELINE. We do not regenerate agent outputs; only the evaluator configuration is changed.

Across these perturbations, the method ordering is retained and the simulation-to-VTSR gap remains. Stricter margins reduce absolute VTSR, looser margins slightly increase VTSR, and the reduced-catalog setting exposes more component-feasibility failures. These results support the robustness of the main conclusion while showing that PE-BENCH remains sensitive to real engineering constraints.

F Baselines and Experimental Protocol

F.1 Baseline ladder

The baselines are designed to separate output-format control, self-refinement, tool use, retry, role decomposition, PE-domain prompting, and PE-structured agent design. The baseline

Table 11: Evaluation-criteria pipeline by converter family. Shared PE criteria are combined with converter-specific physics criteria.

Family	Evaluation sequence
Flyback	required fields and specification alignment; isolation/topology validation; theory consistency criteria for duty, current, reflected voltage, magnetizing inductance, and flux; simulator or formula metrics; BOM and component-feasibility validation; MOSFET/diode/capacitor/core safety-margin; efficiency/ripple/stress/cost criteria; reported-value validation; review/escalation; run record emission.
Buck / Boost / Buck-Boost	required fields and specification alignment; topology feasibility; duty and operating-region criteria; inductor/capacitor sizing; switch/diode/inductor stress; formula metrics; component-database validation and safety-margin; reported-value consistency; performance targets; inconsistency tags and run record emission.
Three-phase inverter	required fields and mode validation; DC-link and modulation feasibility; three-phase voltage/current consistency; switch, diode, gate-driver, and DC-link capacitor validation and safety-margin; THD/current-quality or traction-load consistency; dead-time and shoot-through protection; supported reported values; review/escalation and run record emission.

Table 12: Fixed engineering-inconsistency groups used for analysis.

Inconsistency group	Typical triggers
Data integrity / required format	Missing required fields, invalid JSON, task-candidate mismatch, unparseable numeric units.
Theory / operating mode	Infeasible duty cycle, impossible conversion ratio, CCM/DCM mismatch, wrong topology under isolation requirement.
Component-database validation	Missing or hallucinated part, unsupported component category, missing rating, invalid or unsafe BOM.
Safety-margin / stress	Under-rated MOSFET or diode, capacitor ripple violation, transformer saturation, insufficient margin near a boundary task.
Simulation / formula metrics	Simulation-setup failure, inconsistent simulator configuration, formula metric inconsistency, unsupported waveform-derived metric.
Reported-value consistency	Reported efficiency/ripple/stress/cost exceeds simulator or formula support.
Review/escalation and safety	Ambiguous requirement not escalated, unsafe case not escalated, out-of-scope design signed off as complete.

545 design lets the comparison identify the value of PE-specific evaluation rather than attributing
 546 the result to more attempts or more tools.

547 F.2 Fairness controls

548 All methods receive the same task cards, machine-readable specifications, and bounded
 549 catalog slices. Results record seed, method label, simulation-setup mode, fallback behavior,
 550 evaluator version, simulator calls, runtime, and run-record checksum. Tool and retry baselines
 551 are allowed to use feedback; generic multi-agent baselines are allowed role decomposition;
 552 PE-GPT-STYLE is allowed PE-domain prompting. STRONG BASELINE is our PE-structured
 553 multi-agent system; it is reported only as a reference system for benchmark comparison, not
 554 as part of the evaluator or as a deployable hardware-design system.

555 G Additional Results and Stability

556 Appendix G records the full numeric support behind the main conclusions. All figures and
 557 tables in this section are regenerated or transcribed from the final run logs using the same
 558 method naming as the main text; the PE-structured reference system is labeled STRONG
 559 BASELINE throughout. Each appendix figure has a corresponding numeric table and a short
 560 interpretation.

Table 13: Evaluator validation summary. The suite validates PE-BENCH before leaderboard use by testing required task fields, feasible references, deliberately introduced PE design inconsistencies, independently built valid alternatives, held-out tasks, simulation-setup variation, and data-leakage risk.

Validation family	Cases	Main validation result
Required-field validation	78	All task files parse and expose required fields across the five-family benchmark.
Feasible-reference validation	78	Feasible reference designs satisfy the relevant converter-specific criteria.
Malformed-output validation	500+	Parse and required-format validation identify malformed candidates in the final run record.
Fault-injected design validation	800	99.2% identification rate and 0.0% unsafe false acceptance over injected PE design inconsistencies.
Run-record validation	all runs	Simulation setup, seed, model label, fallback behavior, evaluator version, and run-record checksums are recorded.
Independent-valid-design validation	100	94 accepted; among the six non-accepted alternatives, four are justified review/escalation cases and two are final evaluator false negatives.
Held-out robustness split	6 tasks / 24 run records	STRONG BASELINE VTSR 0.652 versus 0.684 public-dev mean; baseline held-out VTSR 0.158–0.218.
Simulation-setup consistency	60	95.0% decision agreement; median metric errors below 5%.
Contamination / slot-guessing	1000 slots	No data-leakage flags; tolerant guess rates at or below 6.5%.
Synchronous-buck mini-track	8	Extensibility validation: STRONG BASELINE VTSR 0.625 versus best non-specialized VTSR 0.250.

Table 14: Final task-accounting and execution-provenance summary. The table separates the earlier core-only run from the final 78-task run record.

Item	Count	Information recorded in the final artifact
Required-field validation	78/78	All Buck, Boost, Buck–Boost, Flyback, and Three-Phase Inverter task cards parse and expose the required machine-readable fields.
Feasible-reference validation	78/78	Feasible reference designs pass the applicable converter-specific required PE evaluation criteria and serve as feasibility anchors rather than exact-match answers.
Three-phase inverter DC–AC subset inside main benchmark	12 tasks	Real-trace DC–AC tasks with modulation, DC-link, switching-device, dead-time, current-quality/traction-load, reported value, and review/escalation rules.
Synchronous-buck mini-track	8 tasks	Additional extensibility run with dead-time and shoot-through criteria; reported separately from the 78-task leaderboard.
Run source record	all runs	Method label, seed, evaluator version, simulation-setup mode, fallback behavior, simulator calls, runtime statistics, and run-record checksum.
Figure/table source records	all reported plots	Figures and appendix tables are regenerated from frozen final-run summaries using the plotting script released with the artifact.

Table 15: Representative injected engineering inconsistencies.

Injected inconsistency	Detect.	Unsafe accept.	Expected tag
Under-rated MOSFET or diode	100.0%	0.0%	Invalid or Unsafe BOM.
Missing capacitor ripple rating	98.5%	1.5%	Invalid or Unsafe BOM / capacitor ripple violation.
Wrong duty-cycle equation	100.0%	0.0%	Infeasible Theory Failure.
CCM/DCM mismatch	99.2%	0.8%	Operating-Mode Mismatch / Infeasible Theory Failure.
Transformer saturation	100.0%	0.0%	Stress Violation.
Unsupported efficiency value	97.8%	2.2%	Unsupported Performance Value.
Wrong topology under isolation requirement	100.0%	0.0%	Wrong Topology / Review Required.
Unsafe case lacking escalation	98.2%	1.8%	Unsafe case not escalated / Review Required.

Table 16: Expanded independent valid-design validation.

Topology	Alt. candidates	Accepted	Justified re-view/escalation	Final false negatives
Flyback	25	24	1	0
Buck	20	19	1	0
Boost	20	18	1	1
Buck-Boost	15	15	0	0
Inverter	20	18	1	1
Total	100	94	4	2

G.1 Topology slices and held-out robustness

G.2 Reported-value consistency, evaluator validation, and evaluation-criterion inconsistencies

G.3 Final numeric tables behind the figures

G.4 Split, difficulty, retry, and topology-slice tables

The next tables provide the raw numbers behind Appendix Figures 8 and 9. They are kept in the appendix because the main text uses figures to preserve a compact 9-page narrative, while these tables allow reviewers to verify the exact values.

G.5 Analysis, evaluation-criterion, subset, and mini-track tables

The following tables provide the focused analysis subsets behind Appendix Figures 12, 14, 11, and 10. The reported-value consistency table uses a more detailed focused analysis subset than the main leaderboard; the table should therefore be read as a focused analysis, not as a duplicate of the main unsupported-values column.

G.6 Qualitative examples of hidden engineering inconsistencies

Tables and figures show aggregate behavior, while concrete cases show how PE-BENCH distinguishes simulator-executable outputs from verifiable PE designs. Table 34 summarizes three representative cases used during engineering-inconsistency analysis. The cases support the paper’s central conclusions: simulation-only acceptance can miss hidden engineering inconsistencies, protection evidence matters for verifiable task success, and escalating a boundary case can be the correct engineering outcome.

How to read these cases. The case studies connect the aggregate columns in Tables 23, 31, and 33 to the evaluator rules: topology assumptions, equations, BOM ratings, safety

Table 17: Validation-log interpretation of non-accepted independent valid-design candidates.

Non-accepted outcome	Count	Interpretation
Justified review/escalation	4	Candidate can be mathematically plausible but still lack the safety-margin evidence needed for verifiable design consistency.
Final evaluator false negative	2	Candidate appears feasible under the task specification but falls outside a hard threshold, e.g., a creative but mathematically viable soft-switching dead-time margin.
Non-accepted rate	6.0%	Six of 100 alternatives were not accepted: four justified review/escalation cases and two final false negatives.
Final false-negative rate	2.0%	Two of 100 alternatives remain categorized as evaluator false negatives in the final review log.

Table 18: Simulation-setup, held-out-split, and data-leakage validation.

Validation family	Main result	Interpretation
Simulation-setup consistency	60 candidates; 95.0% decision agreement; median errors: efficiency 1.3%, ripple 3.9%, stress margin 4.8%	PE-Bench decisions remain stable across metric pathways.
Held-out split	6 tasks / 24 held-out run records; STRONG BASELINE held-out VTSR 0.652; baseline held-out VTSR 0.158–0.218	Ranking and conclusion persist outside the public-dev task set.
Slot-guessing data-leakage test	1000 hidden slots; exact guesses 1.5–2.5%; tolerant guesses 4.0–6.5%; no flags	The slot-guessing test finds no suspicious memorization pattern.

583 margins, simulation results, reported values, and review/escalation decisions must agree
584 before a candidate is counted as PE-Bench verifiable task success.

585 H Release and Reproducibility

586 H.1 Release layout

587 The anonymized release is organized so that a reviewer can inspect tasks, run required-
588 format validation, evaluate a small candidate subset, and reproduce paper tables
589 from frozen run records. The double-blind review artifact is available at [https://](https://anonymous.4open.science/r/PE-Bench-ACFC/)
590 anonymous.4open.science/r/PE-Bench-ACFC/. The release includes Croissant 1.1 meta-
591 data with Responsible AI conformance (<http://mlcommons.org/croissant/RAI/1.0>) in
592 `croissant_metadata.json`; the metadata describes task records, release manifests, frozen
593 leaderboard summaries, validation summaries, component-catalog slices, schemas, bench-
594 mark/evaluator cards, and responsible-use documentation.

595 H.2 Quickstart

596 The release provides three levels of reproducibility: a fast required-format validation command,
597 a small no-API evaluation run, and a run-record-only reproduction path for all reported
598 tables and figures. The no-API reproduction path verifies frozen manuscript summary
599 records; sanitized API rerun records are included as secondary pipeline evidence and are not
600 used as the manuscript leaderboard source unless explicitly promoted by the alignment gate.
601 When live simulation or magnetics lookup is enabled, the adapters are documented separately
602 from the benchmark criterion logic and can be connected to standard PE simulation and
603 magnetics-design resources [Plexim GmbH, 2026, Martinez, 2024].

Table 19: Evaluator-configuration robustness. The same frozen candidate outputs are re-evaluated under small changes to evaluator tolerances and component-catalog availability.

Configuration	Strong VTSR	Best baseline	Agree.	Decision flips	Rank	Interpretation
Default	0.684	PE-GPT-STYLE 0.252	100.0%	0 (0.0%)	Yes	Reference evaluator configuration used for the main leaderboard.
Stricter margins	0.635	PE-GPT-STYLE 0.228	94.2%	68 P→F; 0 F→P (5.8%)	Yes	Tighter safety and metric checks lower absolute VTSR, while the relative gap remains.
Looser margins	0.702	PE-GPT-STYLE 0.268	96.1%	0 P→F; 46 F→P (3.9%)	Yes	Borderline candidates pass more often, but the method ordering remains stable.
Reduced catalog	0.612	PE-GPT-STYLE 0.205	89.1%	127 P→F; 0 F→P (10.9%)	Yes	Reduced component availability exposes more BOM-feasibility failures without changing the main conclusion.

Table 20: Baseline ladder and what each step isolates.

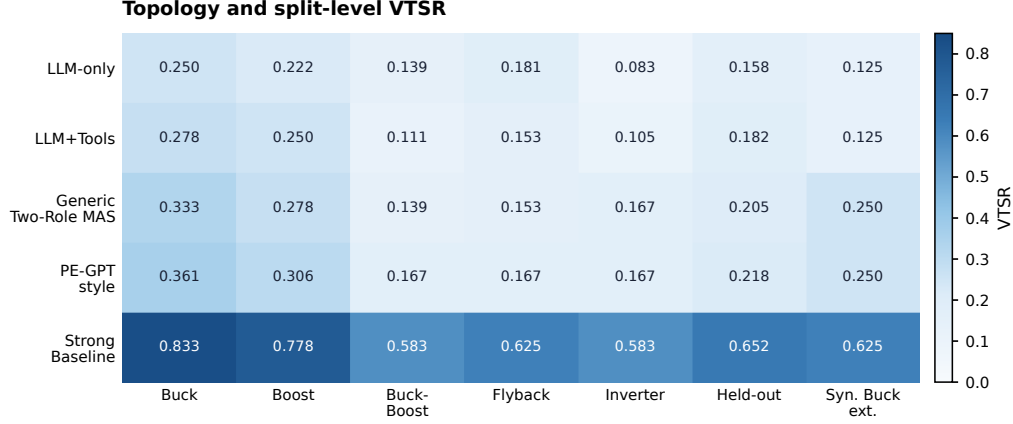
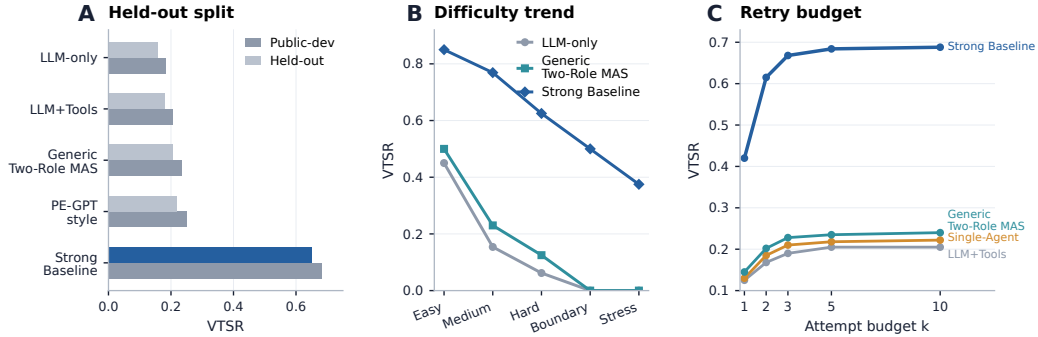
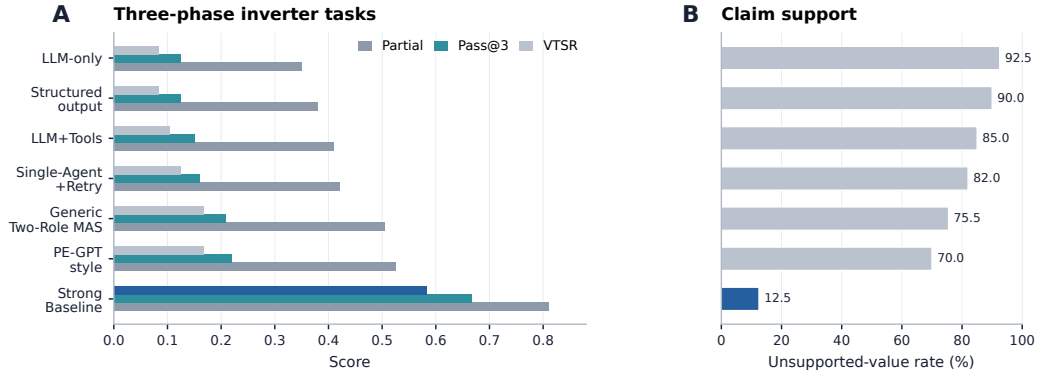
Method	Purpose in the comparison
LLM-only	Tests direct design generation.
Structured-output only	Tests the contribution of output-format compliance.
Text Self-Refine	Tests critique-and-revision.
LLM+Tools	Tests access to calculators and simulator tools.
Single-Agent+Retry	Tests evaluator-feedback retry under a single-agent loop.
Generic Two-Role MAS	Tests generic designer/critic decomposition.
PE-GPT-STYLE	Tests PE-domain prompting inspired by prior PE-GPT-style systems.
STRONG BASELINE	Our PE-structured multi-agent system, reported as a reference system with physical-consistency validation, component-feasibility validation, records of previous inconsistency cases, reported-metric validation, and review/escalation rules.

604 **H.3 Benchmark and evaluator cards**

605 **I Reviewer-Risk Checklist**

Table 21: Runtime and cost accounting used for interpretability rather than as the primary metric.

Method	Tokens/task	Sim calls/task	Wall-clock/task	Est. cost/task
LLM-only	4,500	0.0	12s	\$0.02
LLM+Tools	8,200	2.5	45s	\$0.04
Single-Agent+Retry	12,500	3.0	65s	\$0.06
Generic Two-Role MAS	18,000	3.0	85s	\$0.09
STRONG BASELINE	26,500	1.8	115s	\$0.13

**Figure 8: Topology and split-level VTSR.** The same method ordering is preserved across Buck, Boost, Buck-Boost, Flyback, three-phase inverter, held-out, and synchronous-buck mini-track slices. The heatmap separates core topology coverage from held-out and extensibility validation.**Figure 9: Generalization, difficulty, and retry-budget analyses.** (A) Held-out tasks preserve the ranking with a modest drop for STRONG BASELINE. (B) Difficulty tiers show a smooth trend from easy to stress tasks. (C) More attempts help all methods, and the remaining gap demonstrates the value of PE-structured evaluation.**Figure 10: Three-phase inverter DC-AC subset.** The finalized real-trace DC-AC subset contains 12 grid-connected or traction-mode inverter tasks inside the 78-task benchmark. STRONG BASELINE improves VTSR, Pass@3, and diagnostic partial score while PE-BENCH identifies fewer unsupported reported values.

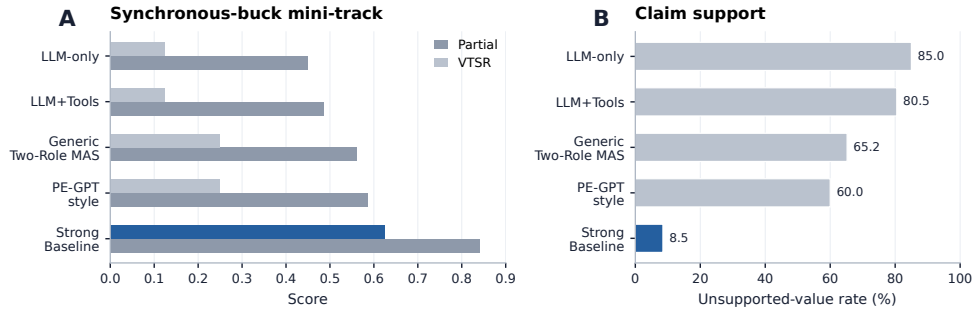


Figure 11: Synchronous-buck mini-track. The eight-task synchronous-buck mini-track validates that the PE evaluation criteria extend beyond the original core families. STRONG BASELINE reaches 0.625 VTSR and PE-BENCH identifies a lower unsupported-reported-value rate.

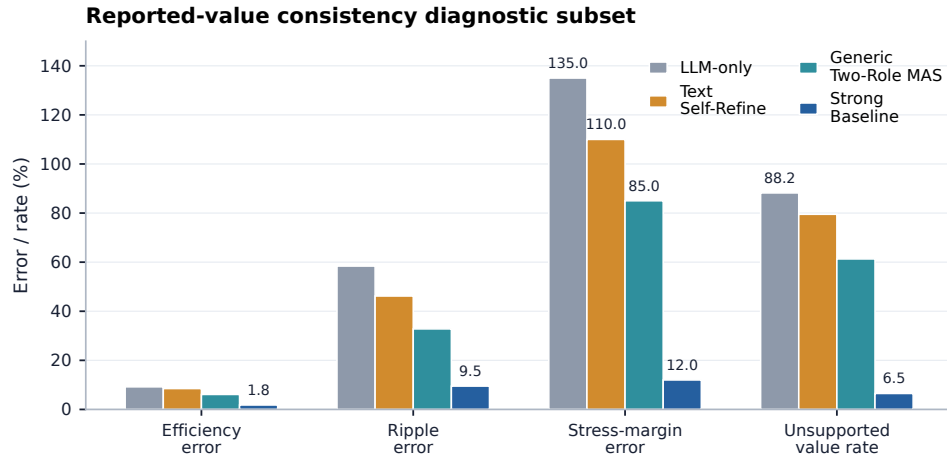


Figure 12: Reported-value consistency focused analysis subset. Unsupported reported values, ripple-error, stress-margin-error, and efficiency-error rates fall when reported performance values are linked to equations, simulation results, component ratings, and safety margins. The focused analysis subset applies a more detailed reported-value consistency criterion than the main leaderboard summary.

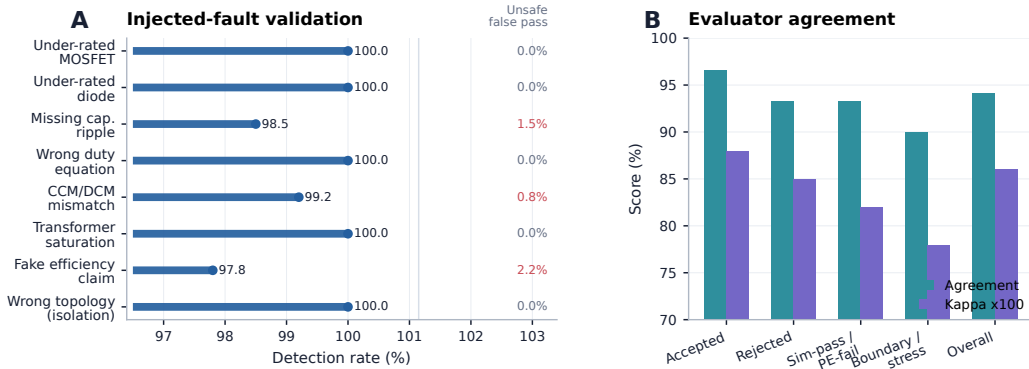


Figure 13: Evaluator validation. (A) Fault-injected validation evaluates whether realistic unsafe near-misses trigger the expected engineering-inconsistency tags without unsafe false acceptances. (B) Aggregated validation results summarize non-reference valid-alternative acceptance, simulation setup decision consistency, held-out-split retention, and the data-leakage test.

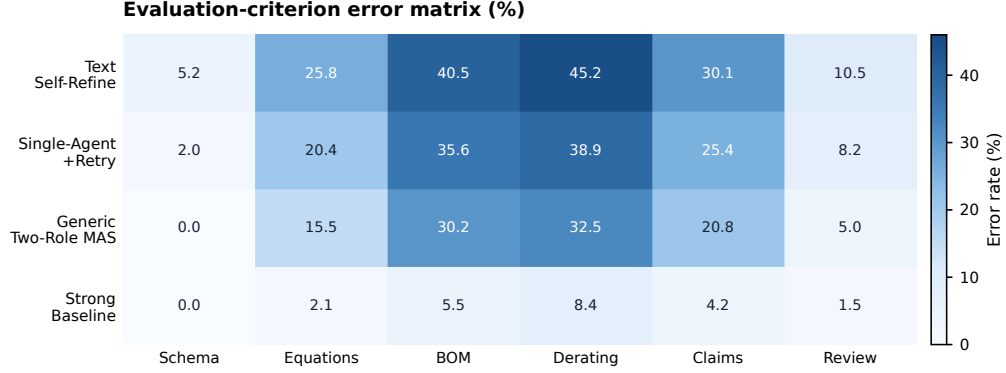


Figure 14: Evaluation-criterion error matrix. Baselines show different inconsistency patterns across required fields, equations, BOM, safety margins, reported values, and review/escalation. STRONG BASELINE reduces error rates across criteria, while the matrix also identifies remaining areas for improvement.

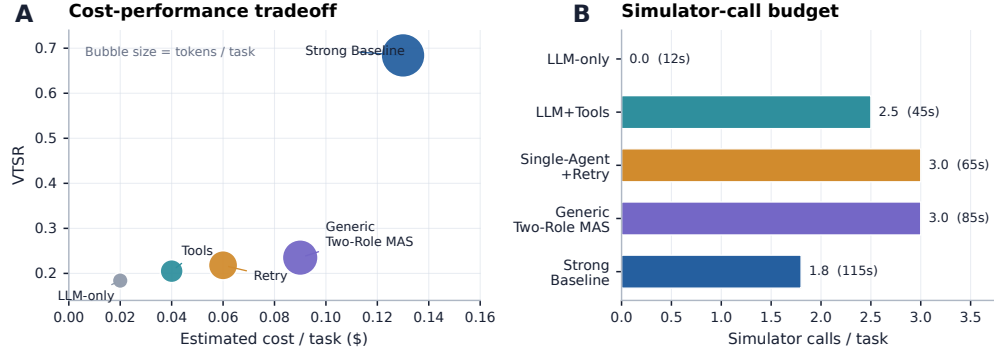


Figure 15: Runtime and cost tradeoff. STRONG BASELINE uses more tokens than direct prompting, uses fewer simulator calls than generic retry-heavy baselines, and produces a much larger VTSR gain. Cost is reported for interpretability, not as the primary objective.

Table 22: Full 78-task leaderboard used in Figure 5.

Method	VTSR	Partial	Pass@3	Unsup. values	Invalid / unsafe BOM	Sim calls
LLM-only	0.184 ± 0.015	0.402	0.201	86.4%	52.1%	0.0
Structured-output only	0.190 ± 0.012	0.415	0.215	82.5%	50.8%	0.0
Text Self-Refine	0.175 ± 0.020	0.422	0.220	76.8%	48.5%	0.0
LLM+Tools	0.205 ± 0.018	0.450	0.245	70.2%	42.0%	2.4
Single-Agent+Retry	0.218 ± 0.022	0.465	0.260	65.5%	39.2%	3.2
Generic Two-Role MAS	0.235 ± 0.019	0.490	0.285	58.0%	35.4%	3.5
PE-GPT-STYLE	0.252 ± 0.025	0.520	0.312	55.4%	30.1%	2.8
Strong Baseline	0.684 ± 0.028	0.845	0.755	8.2%	4.5%	1.7

Table 23: Simulation-only acceptance versus PE-Bench verifiable task success used in Figure 6.

Method	Format valid	Simulator exec.	Sim.-only pass	VTSR	Gap	Main gap issue
LLM-only	97.5%	58.2%	43.5%	0.184	25.1%	Invalid / unsafe BOM
LLM+Tools	98.8%	64.5%	48.0%	0.205	27.5%	Unrealistic Reported value
Generic Two-Role MAS	99.0%	68.1%	52.4%	0.235	28.9%	Stress Violation
PE-GPT-STYLE	100.0%	75.0%	58.5%	0.252	33.3%	Unrealistic Reported value
Strong Baseline	100.0%	94.5%	74.2%	0.684	5.8%	Review Required

Table 24: PE-specific evaluation-criterion ablations used in Figure 7.

Variant	78-task VTSR	Inverter VTSR	Unsupported values	Invalid / unsafe BOM	Unsafe accept.
Full STRONG BASELINE	0.684	0.565	8.2%	4.5%	0.0%
– component-database validation	0.452	0.310	12.5%	46.8%	1.2%
– equation validation	0.188	0.085	45.0%	28.5%	3.5%
– reported-value validation	0.545	0.420	68.4%	8.2%	2.1%
– review/escalation rule	0.612	0.490	10.5%	5.0%	12.8%

Table 25: Backbone robustness used in Figure 7.

Backbone class	LLM-only VTSR	STRONG BASELINE VTSR	Unsupported values	Cost/task
Small / cheap	0.125	0.562	11.5%	\$0.04
Mid general	0.184	0.684	8.2%	\$0.11
Strong general	0.245	0.755	5.4%	\$0.45
Reasoning-oriented	0.312	0.788	4.1%	\$1.20

Table 26: Topology and split-level VTSR used in Figure 8. The synchronous-buck column is an extensibility mini-track and is not included in the 78-task mean.

Method	Buck	Boost	Buck–Boost	Flyback	Inverter	Held-out	Syn. Buck ext.
LLM-only	0.250	0.222	0.139	0.181	0.083	0.158	0.125
LLM+Tools	0.278	0.250	0.111	0.153	0.105	0.182	0.125
Generic Two-Role MAS	0.333	0.278	0.139	0.153	0.167	0.205	0.250
PE-GPT-STYLE	0.361	0.306	0.167	0.167	0.167	0.218	0.250
Strong Baseline	0.833	0.778	0.583	0.625	0.583	0.652	0.625

Table 27: Public-dev versus held-out split. The held-out split evaluates whether the conclusion persists outside the development set rather than measuring a new method.

Method	Public-dev VTSR	Held-out VTSR	Absolute drop	Held-out unsupported values
LLM-only	0.184	0.158	-0.026	88.5%
LLM+Tools	0.205	0.182	-0.023	72.1%
Generic Two-Role MAS	0.235	0.205	-0.030	62.4%
PE-GPT-STYLE	0.252	0.218	-0.034	59.0%
Strong Baseline	0.684	0.652	-0.032	9.5%

Table 28: Difficulty-tier breakdown. Boundary and stress tasks validate whether the submitted evidence supports safe and verifiable design claims; task counts by tier are reported in Table 5.

Method	Easy	Medium	Hard	Boundary	Stress
LLM-only	0.450	0.154	0.062	0.000	0.000
Generic Two-Role MAS	0.500	0.230	0.125	0.000	0.000
Strong Baseline	0.850	0.769	0.625	0.500	0.375

Table 29: Retry-budget sweep. More attempts improve all systems, and PE-structured evaluation maintains a larger VTSR gain with lower regression than generic retry.

Method	$k = 1$	$k = 2$	$k = 3$	$k = 5$	$k = 10$	Regression
LLM+Tools	0.125	0.168	0.190	0.205	0.205	68.4%
Single-Agent+Retry	0.130	0.185	0.210	0.218	0.222	65.2%
Generic Two-Role MAS	0.145	0.202	0.228	0.235	0.240	58.5%
Strong Baseline	0.420	0.615	0.668	0.684	0.688	15.2%

Table 30: Reported-value consistency focused analysis subset. Errors are measured against simulation-backed or formula-backed support in the candidate design output.

Method	Efficiency error	Ripple error	Stress-margin error	Unsupported- value rate
LLM-only	9.2%	58.4%	135%	88.2%
Text Self-Refine	8.5%	46.2%	110%	79.5%
Generic Two-Role MAS	6.1%	32.8%	85%	61.3%
Strong Baseline	1.8%	9.5%	12%	6.5%

Table 31: Evaluation-criterion error rates used in Figure 14. Lower error rates indicate stronger consistency across the required PE evidence.

Method	Required fields	Equations	BOM	Safety margin	Reported values	Escalation
Text Self-Refine	5.2%	25.8%	40.5%	45.2%	30.1%	10.5%
Single-Agent+Retry	2.0%	20.4%	35.6%	38.9%	25.4%	8.2%
Generic Two-Role MAS	0.0%	15.5%	30.2%	32.5%	20.8%	5.0%
Strong Baseline	0.0%	2.1%	5.5%	8.4%	4.2%	1.5%

Table 32: Synchronous-buck mini-track. The mini-track is an extensibility validation outside the 78-task leaderboard, adding dead-time and shoot-through criteria.

Item	Setting
Topology	Synchronous Buck.
Task count	8 tasks: 3 easy, 3 medium, and 2 hard/boundary tasks.
New criteria	Dead-time constraint criterion and shoot-through criterion.
Shared criteria reused	Required fields, requirement interpretation, BOM and component-feasibility validation, safety-margin, reported-value consistency, review/escalation, and run record logging.
Purpose	Validates extensibility of the required PE evaluation criteria beyond the main leaderboard.

Method	VTSR	Partial	Unsupported values	Dominant failure group
LLM-only	0.125	0.450	85.0%	Theory / Data Integrity
LLM+Tools	0.125	0.485	80.5%	Component-database validation
Generic Two-Role MAS	0.250	0.560	65.2%	Performance Requirement
PE-GPT-STYLE	0.250	0.585	60.0%	Simulation Execution Failure
Strong Baseline	0.625	0.840	8.5%	Safety / escalation

Table 33: Three-phase inverter DC–AC subset inside the finalized 78-task benchmark. The inverter tasks use real traces rather than placeholder specifications.

Item	Setting
Topology	Three-phase two-level inverter in grid-connected or traction mode.
Task count	12 tasks: 3 easy, 4 medium, 3 hard, and 2 stress tasks.
Required information	Requirement interpretation; DC-link assumptions; modulation strategy; phase-voltage/current estimates; device and gate-driver BOM; DC-link capacitor rating; simulator configuration; supported reported values; protection and review/escalation decision.
Core criteria	Modulation-index feasibility; DC-link utilization; phase-current and phase-voltage consistency; switch/diode stress and safety-margin; DC-link ripple-current rating; dead-time and shoot-through safety; current-quality or traction-load consistency; reported-value validation; review/escalation.

Method	VTSR	Partial	Pass@3	Unsup. values	Dominant issue
LLM-only	0.083	0.350	0.125	92.5%	Theory / Data Integrity
Structured-output only	0.083	0.380	0.125	90.0%	Theory / Data Integrity
LLM+Tools	0.105	0.410	0.150	85.0%	Component-database validation
Single-Agent+Retry	0.125	0.420	0.160	82.0%	Component-database validation
Generic Two-Role MAS	0.167	0.505	0.208	75.5%	Perf. requirement
PE-GPT-STYLE	0.167	0.525	0.220	70.0%	Simulation Execution Failure
Strong Baseline	0.583	0.810	0.667	12.5%	Safety / escalation

Table 34: Representative qualitative examples. The examples are distilled from final-run inconsistency tags and final-run review notes; the corresponding anonymized task files and candidate outputs are provided with the released artifact.

Case	What the candidate did	PE-Bench result	Why it matters
Simulator-pass with unsupported BOM evidence	A PE-GPT-style flyback candidate produced a complete report and simulator-runnable configuration, but the selected switch margin was inconsistent with the voltage-stress assumptions; the report still presented the design as complete.	Identified component-feasibility and safety-margin inconsistencies; tagged as Invalid or Unsafe BOM / Stress Violation.	Shows how PE-BENCH distinguishes simulator execution from evidence-supported component feasibility: a runnable trace can coexist with an unsupported component choice.
Inverter performance value without protection evidence	An LLM+Tools three-phase inverter candidate satisfied nominal voltage/current calculations but did not provide verifiable protection and DC-link ripple-current support while reporting the design as complete.	Identified protection, DC-link capacitor, and reported-value consistency issues; tagged as Component-database validation / unsafe case not escalated depending on the submitted evidence.	Demonstrates that the inverter DC-AC subset validates the protection and supporting information required for a credible PE design.
Correct escalation on a boundary task	On a boundary/stress requirement with conflicting margins, STRONG BASELINE returned a partial design package and escalated the unresolved assumption instead of fabricating a complete BOM and final efficiency value.	Accepted as a valid review/escalation outcome when the task specification made sign-off unsafe without additional supporting information or tool support; no unsupported success value was counted.	Clarifies that PE-BENCH validates evidence-supported engineering behavior. Escalating a boundary case can be the correct engineering outcome.

Table 35: Recommended release layout.

Path	Contents
tasks/	Machine-readable task cards, machine-readable specifications, split run records, and feasible reference designs.
catalog/	Bounded component-catalog slices or redistributable derived run records.
evaluator/	Evaluation-criteria implementation, inconsistency taxonomy, result format, and simulation setup adapters.
baselines/	Baseline prompts, system wrappers, retry configurations, and PE-MAS configuration.
runs/	Frozen per-task results, suite summaries, seeds, simulation setup run records, evaluator versions, and run-record checksums.
analysis/	Scripts that reproduce paper tables and figures from run records.
docs/	Benchmark card, evaluator card, non-use statement, quickstart, and known limitations.

Table 36: Reviewer-facing reproducibility checklist.

Requirement	Expected support
Anonymous release	Remove usernames, lab paths, private URLs, API keys, and non-anonymous commit run records.
Environment	Provide requirements.txt or environment.yml ; include Dockerfile if practical.
Fast validation	One command parses all task cards and candidate required formats.
Small runnable demo	One command evaluates a small public subset using the formula/stub simulation setup.
Run-record reproduction	One command regenerates the main tables and figure data from frozen result run records.
Croissant metadata	Provide Croissant 1.1 core metadata plus Responsible AI conformance metadata for the task records, evidence summaries, catalogs, schemas, and reviewer-facing cards.
Versioning	Record task version, catalog version, evaluator version, simulation-setup mode, seeds, and run-record checksums.
Licensing	State code/data license and component-catalog redistribution policy.
Non-use statement	State clearly that PE-Bench is not a hardware-certification or product-safety approval tool.

Table 37: Release and evaluator cards.

Item	Description
Release name	PE-BENCH v1.
Intended use	Evaluation and debugging of AI systems for PE design generation.
Not intended for	Certification, production hardware sign-off, regulatory compliance, automated purchasing, or replacement of engineering review.
Evaluator inputs	Candidate JSON, task JSON, catalog slice, simulation setup configuration, and run records.
Evaluator outputs	VTSR, partial scores, constraint violations, metrics, inconsistency tags, execution log, runtime stats, and run-record checksum.
Known abstractions	Formula approximations, simulator simplifications, catalog coverage limits, and future layout/thermal/EMI/certification requirements.
Safety posture	Evidence-based pass criterion; unsafe or ambiguous candidates should be escalated for engineering review or marked as not verifiable.

Table 38: Likely reviewer concerns and where the manuscript addresses them.

Reviewer concern	PE-Bench evidence / mitigation
The benchmark needs a clear evaluation target. The evaluator should recognize non-reference feasible designs.	Appendix A and Appendix B define a reviewer-oriented evaluation target; the full inventory is kept in the artifact. Feasible reference designs are defined as feasibility anchors, and the independent-valid-design test accepts 94/100 non-reference alternatives, with two evaluator false negatives in the final review log.
The evaluator may be biased toward STRONG BASELINE.	Fault-injected, valid-alternative, simulation setup, held-out, and data-leakage tests are independent of our PE-structured reference system.
Simulation-only acceptance should be compared with verifiable task success.	Table 23 shows large gaps between simulation-only acceptance and PE-Bench verifiable task success for baselines.
More attempts or stronger models should be separated from PE evaluation.	Attempt-budget and backbone tables isolate retry and model strength while keeping the PE evaluation criteria fixed.
PE coverage should be stated clearly.	The paper states depth-first scope, includes the inverter DC-AC subset in the 78-task benchmark, and reports the synchronous-buck mini-track separately without asserting full PE coverage.
The release could be misused.	Appendix H includes non-use statements and explicitly excludes certification or production sign-off.

606 NeurIPS Paper Checklist

607 1. Claims

608 Question: Do the main claims made in the abstract and introduction accurately reflect
609 the paper’s contributions and scope?

610 Answer: [\[Yes\]](#)

611 Justification: The abstract and Introduction state the benchmark/evaluator contribution,
612 the 78-task scope, the five converter families, and the main simulation-only versus PE-
613 Bench result. The claims are limited to evaluation of AI-assisted PE design consistency,
614 not hardware certification or deployable power-supply sign-off.

615 Guidelines:

- 616 • The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims
617 made in the paper.
- 618 • The abstract and/or introduction should clearly state the claims made, including the
619 contributions made in the paper and important assumptions and limitations. A [\[No\]](#)
620 or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- 621 • The claims made should match theoretical and experimental results, and reflect how
622 much the results can be expected to generalize to other settings.
- 623 • It is fine to include aspirational goals as motivation as long as it is clear that these
624 goals are not attained by the paper.

625 2. Limitations

626 Question: Does the paper discuss the limitations of the work performed by the authors?

627 Answer: [\[Yes\]](#)

628 Justification: The paper includes an explicit Limitations paragraph in Section 6 and
629 additional scope and non-use statements in Appendices B and H. These sections state
630 that the current benchmark does not cover all PE systems, manufacturing, PCB layout,
631 thermal validation, EMI/EMC, regulatory certification, or replacement of engineering
632 review.

633 Guidelines:

- 634 • The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means
635 that the paper has limitations, but those are not discussed in the paper.
- 636 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 637 • The paper should point out any strong assumptions and how robust the results are
638 to violations of these assumptions (e.g., independence assumptions, noiseless settings,
639 model well-specification, asymptotic approximations only holding locally). The authors
640 should reflect on how these assumptions might be violated in practice and what the
641 implications would be.
- 642 • The authors should reflect on the scope of the claims made, e.g., if the approach was
643 only tested on a few datasets or with a few runs. In general, empirical results often
644 depend on implicit assumptions, which should be articulated.
- 645 • The authors should reflect on the factors that influence the performance of the approach.
646 For example, a facial recognition algorithm may perform poorly when image resolution
647 is low or images are taken in low lighting. Or a speech-to-text system might not be
648 used reliably to provide closed captions for online lectures because it fails to handle
649 technical jargon.
- 650 • The authors should discuss the computational efficiency of the proposed algorithms
651 and how they scale with dataset size.
- 652 • If applicable, the authors should discuss possible limitations of their approach to
653 address problems of privacy and fairness.
- 654 • While the authors might fear that complete honesty about limitations might be used
655 by reviewers as grounds for rejection, a worse outcome might be that reviewers discover
656 limitations that aren’t acknowledged in the paper. The authors should use their
657 best judgment and recognize that individual actions in favor of transparency play
658 an important role in developing norms that preserve the integrity of the community.
659 Reviewers will be specifically instructed to not penalize honesty concerning limitations.

660 3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present new theoretical theorems or proof-based results. The formulas in the paper define evaluation metrics and converter-specific checks rather than formal theoretical claims.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3–5 and Appendices E–H describe the task construction, baseline ladder, evaluator versioning, seeds, splits, run records, validation tests, and figure/table reproduction path. The released artifact is intended to provide task files, evaluator code, frozen run summaries, and analysis scripts needed to reproduce the main reported results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

717 some way (e.g., to registered users), but it should be possible for other researchers
718 to have some path to reproducing or verifying the results.

719 5. Open access to data and code

720 Question: Does the paper provide open access to the data and code, with sufficient instruc-
721 tions to faithfully reproduce the main experimental results, as described in supplemental
722 material?

723 Answer: [Yes]

724 Justification: The submission includes an anonymized release/artifact described in Ap-
725 pendix H, with task cards, evaluator implementation, baseline wrappers, frozen run records,
726 and analysis scripts. The quickstart describes fast validation, small-subset evaluation,
727 and run-record-based reproduction of the reported tables and figures.

728 Guidelines:

- 729 • The answer [N/A] means that paper does not include experiments requiring code.
- 730 • Please see the NeurIPS code and data submission guidelines ([https://neurips.cc/
731 public/guides/CodeSubmissionPolicy](https://neurips.cc/public/guides/CodeSubmissionPolicy)) for more details.
- 732 • While we encourage the release of code and data, we understand that this might not
733 be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not
734 including code, unless this is central to the contribution (e.g., for a new open-source
735 benchmark).
- 736 • The instructions should contain the exact command and environment needed to run
737 to reproduce the results. See the NeurIPS code and data submission guidelines
738 (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- 739 • The authors should provide instructions on data access and preparation, including how
740 to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- 741 • The authors should provide scripts to reproduce all experimental results for the new
742 proposed method and baselines. If only a subset of experiments are reproducible, they
743 should state which ones are omitted from the script and why.
- 744 • At submission time, to preserve anonymity, the authors should release anonymized
745 versions (if applicable).
- 746 • Providing as much information as possible in supplemental material (appended to the
747 paper) is recommended, but including URLs to data and code is permitted.

748 6. Experimental setting/details

749 Question: Does the paper specify all the training and test details (e.g., data splits,
750 hyperparameters, how they were chosen, type of optimizer) necessary to understand the
751 results?

752 Answer: [Yes]

753 Justification: Section 4 defines the baseline ladder, shared task interface, attempt budgets,
754 tool access, metrics, and seed protocol. Appendices F and G provide fairness controls,
755 runtime/cost accounting, held-out and difficulty analyses, topology slices, ablations, and
756 the numeric tables behind the figures.

757 Guidelines:

- 758 • The answer [N/A] means that the paper does not include experiments.
- 759 • The experimental setting should be presented in the core of the paper to a level of
760 detail that is necessary to appreciate the results and make sense of them.
- 761 • The full details can be provided either with the code, in appendix, or as supplemental
762 material.

763 7. Experiment statistical significance

764 Question: Does the paper report error bars suitably and correctly defined or other
765 appropriate information about the statistical significance of the experiments?

766 Answer: [Yes]

767 Justification: The main leaderboard reports VTSR mean and standard deviation across
768 three seeds, and the appendix provides the corresponding numeric tables and additional
769 stability analyses. The paper does not claim a formal hypothesis test; the reported
770 variability is across repeated task-seed evaluations under the stated protocol.

771 Guidelines:

- 772 • The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix F reports tokens per task, simulator calls per task, wall-clock time per task, and estimated cost per task for the compared systems, and the run records store runtime statistics and simulator-call counts. No model training is performed; the experiments are dominated by LLM calls, evaluator execution, and formula/stub/live simulation setup.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work is an evaluation benchmark for AI-assisted PE design, preserves anonymous submission practices, and includes non-use statements that exclude certification, production sign-off, procurement automation, and replacement of qualified engineering review. The paper does not involve private personal data or human-subject experiments.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

828 Question: Does the paper discuss both potential positive societal impacts and negative
829 societal impacts of the work performed?

830 Answer: [Yes]

831 Justification: The paper discusses positive impacts through more rigorous evaluation
832 of AI-assisted PE design for important electrified systems, and it discusses negative or
833 misuse risks through the limitations and non-use statements. In particular, it states
834 that PE-Bench is not a hardware-certification, regulatory-compliance, or product-safety
835 approval tool.

836 Guidelines:

- 837 • The answer [N/A] means that there is no societal impact of the work performed.
- 838 • If the authors answer [N/A] or [No], they should explain why their work has no societal
839 impact or why the paper does not address societal impact.
- 840 • Examples of negative societal impacts include potential malicious or unintended uses
841 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g.,
842 deployment of technologies that could make decisions that unfairly impact specific
843 groups), privacy considerations, and security considerations.
- 844 • The conference expects that many papers will be foundational research and not tied to
845 particular applications, let alone deployments. However, if there is a direct path to any
846 negative applications, the authors should point it out. For example, it is legitimate to
847 point out that an improvement in the quality of generative models could be used to
848 generate Deepfakes for disinformation. On the other hand, it is not needed to point
849 out that a generic algorithm for optimizing neural networks could enable people to
850 train models that generate Deepfakes faster.
- 851 • The authors should consider possible harms that could arise when the technology is
852 being used as intended and functioning correctly, harms that could arise when the
853 technology is being used as intended but gives incorrect results, and harms following
854 from (intentional or unintentional) misuse of the technology.
- 855 • If there are negative societal impacts, the authors could also discuss possible mitigation
856 strategies (e.g., gated release of models, providing defenses in addition to attacks,
857 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
858 feedback over time, improving the efficiency and accessibility of ML).

859 11. Safeguards

860 Question: Does the paper describe safeguards that have been put in place for responsible
861 release of data or models that have a high risk for misuse (e.g., pre-trained language
862 models, image generators, or scraped datasets)?

863 Answer: [N/A]

864 Justification: The paper does not release a high-risk pretrained model, image generator,
865 scraped personal dataset, or other asset with the misuse profile described in the question.
866 The benchmark release still includes conservative pass criteria, review/escalation handling
867 for unsafe or ambiguous cases, and non-use statements for responsible interpretation.

868 Guidelines:

- 869 • The answer [N/A] means that the paper poses no such risks.
- 870 • Released models that have a high risk for misuse or dual-use should be released with
871 necessary safeguards to allow for controlled use of the model, for example by requiring
872 that users adhere to usage guidelines or restrictions to access the model or implementing
873 safety filters.
- 874 • Datasets that have been scraped from the Internet could pose safety risks. The authors
875 should describe how they avoided releasing unsafe images.
- 876 • We recognize that providing effective safeguards is challenging, and many papers do
877 not require this, but we encourage authors to take this into account and make a best
878 faith effort.

879 12. Licenses for existing assets

880 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
881 the paper, properly credited and are the license and terms of use explicitly mentioned
882 and properly respected?

883 Answer: [Yes]

884 Justification: Existing assets and references used by the paper, including PE-GPT-style
885 work, EngDesign, PLECS, OpenMagnetics, and IEA reports, are cited in the References.
886 The IEA reports are marked with the stated CC BY 4.0 license, and Appendix H includes
887 the release licensing and component-catalog redistribution policy as part of the artifact
888 checklist.

889 Guidelines:

- 890 • The answer [N/A] means that the paper does not use existing assets.
- 891 • The authors should cite the original paper that produced the code package or dataset.
- 892 • The authors should state which version of the asset is used and, if possible, include a
893 URL.
- 894 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 895 • For scraped data from a particular source (e.g., website), the copyright and terms of
896 service of that source should be provided.
- 897 • If assets are released, the license, copyright information, and terms of use in the
898 package should be provided. For popular datasets, `paperswithcode.com/datasets`
899 has curated licenses for some datasets. Their licensing guide can help determine the
900 license of a dataset.
- 901 • For existing datasets that are re-packaged, both the original license and the license of
902 the derived asset (if it has changed) should be provided.
- 903 • If this information is not available online, the authors are encouraged to reach out to
904 the asset’s creators.

905 13. New assets

906 Question: Are new assets introduced in the paper well documented and is the documenta-
907 tion provided alongside the assets?

908 Answer: [Yes]

909 Justification: The paper introduces new benchmark/evaluator assets: machine-readable
910 PE tasks, feasible reference designs, evaluator criteria, failure tags, baseline wrappers, and
911 frozen run records. Appendices B–H document the benchmark card, task fields, evaluator
912 interface, validation suite, experimental protocol, release layout, quickstart, limitations,
913 and intended/non-intended uses.

914 Guidelines:

- 915 • The answer [N/A] means that the paper does not release new assets.
- 916 • Researchers should communicate the details of the dataset/code/model as part of their
917 submissions via structured templates. This includes details about training, license,
918 limitations, etc.
- 919 • The paper should discuss whether and how consent was obtained from people whose
920 asset is used.
- 921 • At submission time, remember to anonymize your assets (if applicable). You can either
922 create an anonymized URL or include an anonymized zip file.

923 14. Crowdsourcing and research with human subjects

924 Question: For crowdsourcing experiments and research with human subjects, does the
925 paper include the full text of instructions given to participants and screenshots, if
926 applicable, as well as details about compensation (if any)?

927 Answer: [N/A]

928 Justification: The paper does not involve crowdsourcing, user studies, paid participants,
929 or human-subject data collection. The review/escalation decision is an evaluator field for
930 unsafe or ambiguous engineering cases, not a study involving human participants.

931 Guidelines:

- 932 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
933 with human subjects.
- 934 • Including this information in the supplemental material is fine, but if the main contri-
935 bution of the paper involves human subjects, then as much detail as possible should
936 be included in the main paper.
- 937 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
938 or other labor should be paid at least the minimum wage in the country of the data
939 collector.

940 **15. Institutional review board (IRB) approvals or equivalent for research with**
941 **human subjects**

942 Question: Does the paper describe potential risks incurred by study participants, whether
943 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
944 approvals (or an equivalent approval/review based on the requirements of your country or
945 institution) were obtained?

946 Answer: [N/A]

947 Justification: The paper does not involve crowdsourcing or research with human subjects,
948 so IRB approval or equivalent human-subject review is not applicable. No participant
949 instructions, compensation, or participant-risk disclosures are involved.

950 Guidelines:

- 951 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
952 with human subjects.
- 953 • Depending on the country in which research is conducted, IRB approval (or equivalent)
954 may be required for any human subjects research. If you obtained IRB approval, you
955 should clearly state this in the paper.
- 956 • We recognize that the procedures for this may vary significantly between institutions
957 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and
958 the guidelines for their institution.
- 959 • For initial submissions, do not include any information that would break anonymity (if
960 applicable), such as the institution conducting the review.

961 **16. Declaration of LLM usage**

962 Question: Does the paper describe the usage of LLMs if it is an important, original, or
963 non-standard component of the core methods in this research? Note that if the LLM
964 is used only for writing, editing, or formatting purposes and does *not* impact the core
965 methodology, scientific rigor, or originality of the research, declaration is not required.

966 Answer: [Yes]

967 Justification: LLMs are central to the evaluated systems and baselines, and Section 4
968 describes LLM-only, structured-output, self-refine, tool-using, retry, generic multi-role,
969 PE-GPT-style, and Strong Baseline systems. The paper uses LLMs as evaluated design-
970 generation components, not merely for writing or formatting.

971 Guidelines:

- 972 • The answer [N/A] means that the core method development in this research does not
973 involve LLMs as any important, original, or non-standard components.
- 974 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
975 not be described.